Future Generation Computer Systems 174 (2026) 107952



Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs





One sketch for all super hosts: Large-scale network monitoring with UnivCar

Jiqiang Xia [a], Le Tian [a,b,*], Yuxiang Hu [a,b], Jianjin Zhao [c], Penghao Sun [d], Jianhua Peng [e]

[a] Information Engineering University, Zhengzhou, China
[b] National Key Laboratory of Advanced Communication Networks, Zhengzhou, China
[c] Beijing University of Posts and Telecommunications, Beijing, China
[d] Academy of Military Sciences, Beijing, China
[e] Purple Mountain Laboratories, Nanjing, China

ARTICLE INFO

Keywords:
Network monitoring
Super host identification
Sketch
Anomaly traffic detection
Cardinality estimation

ABSTRACT

A host that engages in an unusually high level or significant change of distinct connectivities is recognized as a super host, which typically correlates with an abnormal event in the network. With the rapid expansion of network scale and traffic volume, accurately measuring host cardinalities to identify super hosts has become increasingly challenging. Sketch-based approaches, known for their efficient compressed recording structure and provable error bounds, emerge as a promising method for estimating host cardinalities. However, existing solutions encounter two challenges: 1) inefficient memory utilization of estimating host cardinalities with skewed distribution; 2) insufficient capacity and flexibility to simultaneously measure different types of host cardinalities. To address these problems, we propose a novel sketch, named UnivCar, to efficiently estimate skewed host cardinalities for identifying various super hosts among large-scale network traffic. UnivCar utilizes a hierarchical data structure that supports adaptive extensions to enable memory-efficient tracking on the highly skewed cardinality distribution. To ensure the cardinality estimation accuracy of various super hosts, we present a lossless extension algorithm that enables flexible switching between different estimation tasks. We provide a theoretical analysis and perform a series of evaluations on UnivCar based on real-world traffic traces. Experimental results show that UnivCar outperforms existing sketch-based approaches in both accuracy and performance for identifying various super hosts while consuming less memory.

1. Introduction

Host cardinality estimation is an essential task in network measurement that significantly contributes to both network monitoring and malicious traffic detection [1,2]. The cardinality of a host refers to the number of distinct peers it connects with. Specifically, the destination cardinality represents the number of distinct destination addresses emitted from a source host, while the source cardinality reflects the number of distinct source addresses connected to a destination host.

A host that shows abnormally high level or significant change of host cardinalities is identified as a super host, usually associated with an abnormal event in the network. For instance, if a host presents high source cardinality, it may be experiencing a Distributed Denial of Service (DDoS) attack, and can be recognized as a super receiver [3,4]. On the contrary, a high destination cardinality suggests that the host could be a super spreader due to a worm infection [5,6]. Additionally, a super changer is a host that displays significant changes in cardinality between two adjacent time intervals, which usually illustrates the existence of anomaly traffic [7,8]. Therefore, accurate identification of super hosts can help network administrators detect malicious attacks (e.g., DDoS attacks, network scanning, worm propagation) in a timely manner, which is critical for network security operation and management [9]. Moreover, super host identification can also be utilized in other practical applications, including recommendation system [10], social networks [11–13], and search trend detection [14,15].

However, considering the dramatically increased scale and complexity of modern network traffic [16–18], it is challenging to monitor tremendous ongoing flows and recognize super hosts among them. In this context, sampling-based approaches [19,20] are commonly used for network measurement due to their simplicity and scalability. However, these methods often struggle with measurement accuracy and can introduce additional communication overhead with remote sampling collectors. Sketch-based approaches are yet another promising option for super host identification by estimating host cardinality [2,21,22]. In general, these solutions compact the statistical features of large-scale traffic into an approximate data structure with theoretical provable error bound. This achieves an effective trade-off between

* Corresponding author.
E-mail address: Le.tian2019@outlook.com (L. Tian).

https://doi.org/10.1016/j.future.2025.107952
Received 10 February 2025; Received in revised form 18 May 2025; Accepted 31 May 2025
Available online 17 June 2025
0167-739X/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

memory efficiency and estimation accuracy. Sketch-based approaches have proven to be highly efficient in identifying super receivers, super spreaders, and super changers. However, there are still many problems that remain to be solved for super host identification.

First, most previous works fail to provide flexible enough data structure for estimating host cardinalities with skewed distribution. The host cardinalities of real network traffic conform to a heavy-tailed distribution [7,14,23]. A small number of super hosts possess high cardinalities, while most flows are associated with normal hosts with small cardinality. However, most existing sketch-based solutions use a fixed-size data structure to estimate different traffic traces with skewed cardinality distributions [3,8,24]. The data structures of these methods have to be large enough to ensure the estimation accuracy of high cardinality hosts. In this case, significant memory resources are wasted in monitoring large numbers of low cardinality hosts, which is meaningless for super host identification.

Second, estimating source and destination cardinalities simultaneously is essential for tracking anomaly hosts but is not supported by most previous works. Over the past decades, network attacks increasingly manifest as a sophisticated combination of multiple types rather than relying on a single tactic [25–27]. For example, a classic DDoS attack usually relies on a scanning attack to capture potential targets and then launch the flooding traffic against a specific target to maximize attack effects [28]. Effectively detecting this typical attack chain firmly hinges on the simultaneous identification of various super hosts [8,29]. Super hosts often present abnormal behaviors regarding their source and destination cardinalities. However, it is striking that most previous approaches can only monitor a single type of host cardinality. To enable various cardinalities estimation, a naive method is to allocate each cardinality estimation task with an individual memory space. While efficient and easy to implement, this method can result in significant wastage of memory resources. Meanwhile, a practical super host identification system should flexibly shift its measurement attention towards different host cardinalities for different network states.

In this paper, we propose a novel sketch, named UnivCar, to efficiently estimate skewed host cardinalities for identifying various super hosts among large-scale network traffic. UnivCar is generally constructed from a hierarchical data structure consisting of the filtering and detecting parts to enable memory-efficient tracking on the highly skewed cardinality distribution. The filtering part eliminates tremendous small cardinality flows that have no association with super hosts. The detecting part is extensible and supports the efficient identification of high cardinality flows correlated with potential super hosts. Based on the proposed lossless extension algorithm, UnivCar can dynamically adjust its data structure to simultaneously and flexibly perform different cardinality estimation tasks. Before and after extending its data structure, UnivCar persistently supports the efficient reconstruction of the IP addresses for the identified super hosts. We highlight the main contributions of our work as follows.

(1) We propose a novel sketch, named UnivCar, to simultaneously identify different types of super hosts among large-scale network traffic. The extensible data structure of UnivCar enables memory-efficient estimation of different host cardinalities with skewed distribution;
(2) We propose a lossless extension algorithm to flexibly shift the measurement attention of UnivCar towards different host cardinalities once the network state changes;
(3) We present a theoretical analysis of the error bounds and complexities of the UnivCar sketch;
(4) Based on real-world traffic traces, we evaluate the detection accuracy and performance of the UnivCar sketch compared with the state-of-the-art solutions.

The rest of this paper is organized as follows. Section 2 introduces related works on super host identification. Section 3 provides the design overview of our proposed UnivCar sketch, including design rationale and details of the data structure and operations. Section 4 presents a theoretical analysis on the estimation error bounds and complexity of UnivCar sketch. Section 6 presents the experimental investigation and Section 8 gives the conclusion of this paper.

2. Related works

Large-scale network monitoring becomes extremely difficult considering the dramatically increased traffic volume [30]. Existing approaches explore the compromises between measurement granularity and operational overhead, primarily classified as three paradigms: Sampling-based solutions attempt to probabilistically reduce the size of measured traffic by slicing the original traffic, trading reduced processing load against risks of information loss; Machine learning-based solutions extract behavioral patterns through automated feature learning, requiring significant computing resources and labeled datasets; Sketch-based solutions perform approximate processing with theoretically provable error bounds, achieving the trade-off between resources consumption and measurement accuracy while preserving the size of the original traffic.

2.1. Sampling-based solutions

Many sampling-based measurement systems [31–34] are developed by monitoring only a fraction of flows (i.e., the sampling probability) to infer the cardinalities of sampled flows. NetFlow [19] and sFlow [20] are two classical sampling-based solutions that have been integrated as measurement tools into the operating system of forwarding devices. However, both of them lack support for super host identification. Venkataraman et al. [33] propose the first sampling-based algorithm for recognizing superspreaders from high-speed traffic. Cao et al. [34] propose an online sampling-based approach for super host identification. They adopt the two-phase filtering method to eliminate low cardinality hosts, significantly reduce the memory overhead.

To increase the detection accuracy of previous sampling-based approaches, some non-duplicated sampling solutions [35,36] are proposed which samples only the first occurrence of an element in each flow and block it on subsequent occurrences. These approaches are efficient but not suitable for a sampling probability that is either too small or too big. In general, sampling-based solutions can significantly reduce packet processing overhead both in time and space, but inevitably sacrifices accuracy and misses important events since only the cardinality of sampled hosts can be estimated. Moreover, it is difficult to simultaneously measure various types of cardinalities by employing sampling-based methods.

2.2. Machine learning-based solutions

Machine learning-based network traffic monitoring systems have dramatically evolved in past decades. Early approaches employ manual feature engineering (flow duration, packet size distribution) with classical classifiers (Random Forest [37], Support Vector Machine (SVM) [38]), but encounter scalability limitations in feature computation/storage and inadequate modeling of network traffic's spatiotemporal dependencies. Subsequent deep learning paradigms address these issues through automated feature extraction from raw traffic data: Convolutional Neural Networks process packet-level byte sequences [39], Linear SVMs capture temporal patterns [40], and Graph Neural Networks integrate topological relationships for anomaly node detection [41]. Nevertheless, these models incur substantial computational overhead, causing prohibitive training or inference latency in high-speed networks requiring real-time responsiveness. Extensions of these methodologies demonstrate cross-domain applicability in attack traceability [42] and anomalous image detection [43].

Table 1
Capacities of different super host identification approaches.

	SCE/DCE	SCE+DCE	SCD	Rev	LS
VBF [24]	✓	×	×	✓	×
Zhang et al.[6]	✓	×	✓	×	×
INDDoS [3]	✓	×	×	×	×
ExtendedSketch [7]	✓	×	✓	✓	×
DCDS [50]	✓	×	×	✓	×
SuperSketch [8]	✓	✓	×	✓	×
Zhou et al.[29]	✓	✓	✓	✓	×
SpreadSketch [5]	✓	×	✓	✓	×
UnivCar	✓	✓	✓	✓	✓

SCE: Source Cardinality Estimation; DCE: Destination Cardinality Estimation.
SCD: Skewed Cardinality Distribution; Rev: Reversibility.
LS: Lossless Switching between different estimation tasks.

Furthermore, some frameworks including Online Learning [44] and Reinforcement Learning [45] adopt persistent model updates to handle dynamically changed traffic patterns. However, these approaches exhibit inherent limitations in their computational intensity or dependency on labeled data, significantly limiting their scalability in large-scale network monitoring scenarios. Some recently proposed learning models may address these issues: Kolmogorov-Arnold Networks [46,47] reduce parameter reliance and computing complexity, while Segment Anything Model [48,49] demonstrates zero-shot capability with minimal labeled data. However, their adaptation to network traffic analysis remains unexplored.

2.3. Sketches for host cardinality estimation

In contrast to both sampling's information loss and learning methods' resource demands, sketch-based solutions achieve a unique balance. They preserve critical traffic statistics through compact data structures while avoiding computationally intensive model training. With the rapid growth in network traffic scale, sketch-based solutions are becoming one of the most promising network measurement methods. A series of sketch-based approaches [3,7,50,51] are proposed to identify super hosts by estimating source/destination cardinalities. INDDoS [3] attempts to combine the Count-Min sketch [52] and Direct Bitmap [53] to identify the DDoS victims by estimating the source cardinality of each host. Each host is allocated with the same size of memory space for source cardinality estimation. This method encounters significant challenge in memory consumption since the per-flow cardinality distribution is often highly skewed. To address this problem, ExtendedSketch [7] provide an extensible sketch to dynamically adjust the size of memory allocated to different hosts. It achieves a great trade-off between memory utilization efficiency and super host identification efficiency.

However, most of the previous sketch-based solutions do not support estimate the source and destination cardinality simultaneously which preserving non-trivial effect on network security. A few of approaches [8,29] are proposed to address this problem in recent years. Jing et al. [8] propose a multi-dimensional reversible data structure, named SuperSketch, that enables simultaneously measuring source cardinality, destination cardinality, and destination port cardinality. Zhou et al. [29] propose a distributed network monitoring system which can estimate the source and destination cardinalities simultaneously and adjust the sketch size to meet large cardinalities monitoring demands. However, these approaches allocate individual memory space for the estimation of different host cardinalities, causing an inevitable waste of memory resources when only one kind of super host exists in the network. Besides, these methods are not capable for flexible switching between different estimation tasks (see Table 1).

Table 2
Notations in this paper.

Notation	Description
T_e	Threshold for expeling an item from the filtering part
T_{rcv}	Threshold for identifying super receiver
T_{spr}	Threshold for identifying super spreader
T_{ch}	Threshold for identifying super changer
w	Bucket number of the filtering part
m	Bit size of each bitmap cell in the filtering part
d	Sketch depth of the detecting part
W_i	Bucket number of the ith row in the detecting part, $i \in \{1, 2, \dots, d\}$
M	Bit size of each bitmap cell in the detecting part
h_{bit}	Hash function for bit-locating in the filtering part
h_{bkt}	Hash function for bucket-selection in the filtering part
H_{bit}	Hash function for bit-locating in the detecting part
H^i_{bkt}	A hash function for bucket-selection in the detecting part, $i \in \{1, 2, \dots, d\}$
$\mathcal{A}[h]$	Estimated cardinality of host h.
θ	Threshold for the bitmap extension in each bucket.
λ	Threshold for the bucket extension in each row.

2.4. Sketches for generic estimation tasks

Recently, generic measuring sketch becomes a promising means in network monitoring including FermatSketch [54], UnivMon [55], Elastic Sketch [56], CountLess sketch [57], Joltik [58] and ActiveCM [59]. These methods accomplish multiple estimation tasks (i.e., flow size, heavy hitter, entropy etc.) within one data structure to improve measuring efficiency and flexibility. UnivMon [55] is one of the most typical generic measuring sketches which provides a memory-efficient method to compute per-flow frequency. The following works named Joltik [58] and ActiveCM [59] improve UnivMon both in processing speed and memory efficiency.

However, the estimation of flow frequency distribution or the per-flow cardinality statistics are not considered in these solutions. As a complement to existing generic measuring sketches, Zhang et al. [6] adopt the adjustable progressive sampling technique, which samples subsets of flows by an exponentially decreasing probability according to their cardinalities. It improves the memory efficiency of cardinality estimation with skewed distribution, achieving lower memory cost than UnivMon [55] and ActiveCM [59].

There are also many configurable sketches attempt to present dynamic sketch algorithms that comprehensively consider many factors including resource limitation, traffic distribution, estimation accuracy and efficiency. DISCO [60] is a novel sketch framework that can be dynamically configured in terms of the number of hash functions to optimize the estimation accuracy. It can be applied to three classical solutions, including the Count-min [52], Conservative Update(CU) [61], and Count sketches [62]. In contrast, Namkung et al. focus on the hardware implementation of the sketch algorithm to optimize its resource utilization. They propose the Sketchlib to achieve efficient deployment of sketch algorithms in programmable switches while considering hardware resource limitations. They evaluate the utility of SketchLib in many applicable sketches including Count Sketch [62], UnivMon [55], and Elastic Sketch [56].

3. UnivCar design

In this section, we detail the design of UnivCar sketch. First, we analyze the shortcomings of existing solutions and explain the design rationale of UnivCar. Second, we present a detailed illustration of its hierarchical and extensible data structure. Third, we describe its primary operations: update, extension, query, and reconstruction. The notations in this paper are listed in Table 2.

3.1. Rationale

As shown in Fig. 1, the UnivCar sketch generally consists of two parts. The filtering part aims to filter massive flows with low host cardinalities, while the detecting part further estimates the high cardinalities of potential super hosts. The design of UnivCar sketch derives from the following practical premises.

A hierarchical and extensible data structure for skewed cardinality distribution. In most cases, the network traffic consists of a large number of flows with low host cardinalities. On the contrary, flows transferred towards/from a super host tend to possess high host cardinalities. For example, DDoS attack victims possess large source cardinalities, while super spreaders are often associated with large destination cardinalities. We need to construct a large enough bitmap register for each destination or source node to preserve accuracy when estimating the large-scale traffic containing anomaly flows. This could be a significant waste of the precious memory resources of a measuring node. To improve the memory efficiency of cardinality estimation, UnivCar utilizes a hierarchical and extensible data structure to encounter skewed cardinality distributions. The filtering part consists of small scale but large number of bitmaps to filter giant normal flows with low host cardinalities. On the contrary, the detecting part uses multiple large bitmaps to estimate high host cardinalities accurately. Moreover, it can also extend the bitmaps and buckets based on our proposed extension algorithm (Algorithm 1) to improve estimation accuracy and efficiency.

Simultaneous and flexible estimation of various host cardinalities. Although a few solutions [8,29] can realize concurrent execution of different cardinality measurement tasks, they typically need to allocate independent memory space for each measurement task, resulting in the waste of memory resources. Meanwhile, an efficient and practical estimation solution is supposed to flexibly shift its measurement attention towards different host cardinalities once the network state changes. Existing solutions have insufficient or no support for switching between different measurement tasks. To address these problems, UnivCar estimates the source and destination cardinalities in the same bucket (estimation unit of a sketch). Each bucket of the detecting part uses a sharing-exclusive strategy to enable flexible switching between different cardinality measurement tasks. Specifically, UnivCar estimates source and destination cardinalities in parallel at the beginning of each measurement epoch. These two cardinality estimation tasks both share half of the bitmap registers of each bucket, which is called sharing mode. Once the estimated value of the source/destination cardinality exceeds a predefined threshold, the corresponding bucket will transfer to the exclusive mode. In this case, one cardinality estimation task possesses the whole memory resource of the bucket. Besides, we also propose an extension algorithm (Algorithm 1) to enable lossless switching from the sharing mode to the exclusive mode without reconfiguring the sketch.

Efficient reconstruction of super host addresses. Reversibility is another crucial capability for super host identification. Once a super host detected, timely reconstruction of the host address is required to enable efficient defense towards that anomaly host. However, many solutions fail to achieve efficient reconstruction operations. The difficulty arises from the compressed data structure used in sketch algorithms. Existing reversible sketches often encounter additional memory access [7] or computing overhead [50,63], reducing packet processing throughput or causing inevitable recovery delays. To address this problem, we propose a reconstruction algorithm (Algorithm 2) and carefully design the data structure of the detecting part based on the Chinese Reminder Theory (CRT). Meanwhile, we also present a theoretical analysis to illustrate that the algorithm can effectively reconstruct host addresses both before and after the data structure expansion.

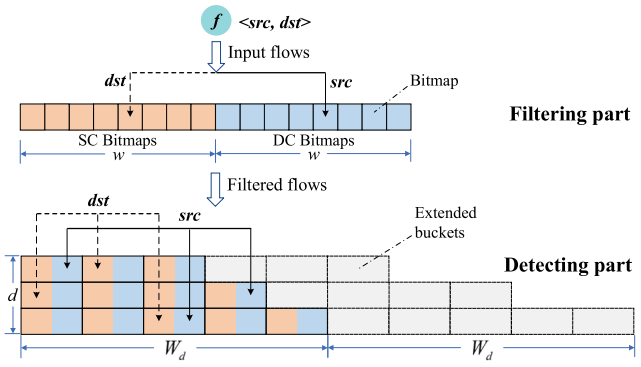


Fig. 1. Data structure of UnivCar sketch.

3.2. Data structure

The filtering part consists of two similar parts with w buckets in one row. One is for source cardinality estimation, and the other is for destination cardinality estimation. Since cardinality estimation requires eliminating duplicate elements, each bucket of the filtering part consists of a classical m sized bitmap register. If the value of a bit is 1, it indicates that at least one flow is hashed to that location, and 0 indicates that none of the flows are mapped to that location. We estimate the source/destination cardinality of a host by computing the sum of values in the bitmap register (i.e., the number of 1s). Suppose the estimated value of the filtering part is greater than a predefined threshold T_e. In that case, we expel the corresponding item from the bucket and sign it as a potential super host, which will be further estimated in the detecting part.

The detecting part simultaneously estimates source cardinality and destination cardinality in each bucket and supports flexible switching between them. It follows the basic structure of a d depth CM sketch [52] with W_i buckets in each row($i \in \{1, 2, \ldots, d\}$). To efficiently apply the Chinese Remainder Theorem to reconstruct super host address, $W_1, W_2, \ldots, W_d$ are set as pairwise mutually prime numbers. Each bucket consists of two M sized bitmaps and the corresponding flags ($flag_src$ and $flag_dst$) that indicate what kind of cardinality is being estimated. In the initial phase of the measurement, the estimation of the source cardinality and the destination cardinality is conducted simultaneously in each bucket of the detecting part. These two measurement tasks are performed in the *sharing mode*, each occupying half of the bitmap resources. When the estimated value of one of the tasks exceeds a threshold(i.e., θM), we switch the cardinality estimation in that bucket to the exclusive mode, measuring only the source or destination cardinality. Moreover, the detecting part also performs bucket extension operations to avoid frequent hash collisions when the extended bitmaps account exceeds the predefined threshold λW_i (detailed in Section 3.3.2).

Fig. 2 demonstrates the processing workflow of UnivCar through a concrete case study involving three flow categories: normal low-cardinality flows (gray), normal high-cardinality flows (blue), and abnormal flows from super hosts (red). We set the thresholds as $T_e = 10$, $T_{rcv} = 80$, and $\theta = 1/2$. For all the inputting flows, the filtering part of UnivCar simultaneously estimates the source and destination cardinalities. The majority of normal flows with cardinalities less than T_e are retained and estimated within the filtering part. In contrast, the high-cardinality flows exceeding the T_e threshold are expelled to the detecting part for further estimation. In the detecting part, the estimated source and destination cardinality of the normal high cardinality flow (blue) are 27 (minimum of 29,31,27) and 14 (minimum of 17,22,14), respectively. Notably, these values remain below the bitmap extension threshold $\theta M = 32$ (derived from $\theta = 1/2$ applied to a 64-bit bitmap register), thus preventing bitmap expansion in the

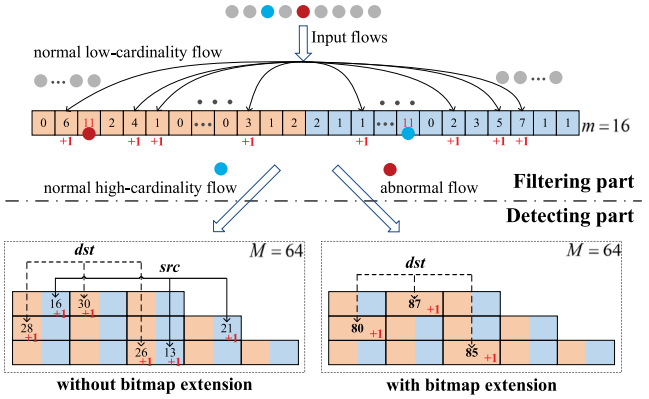


Fig. 2. An example of the processing logic in UnivCar.

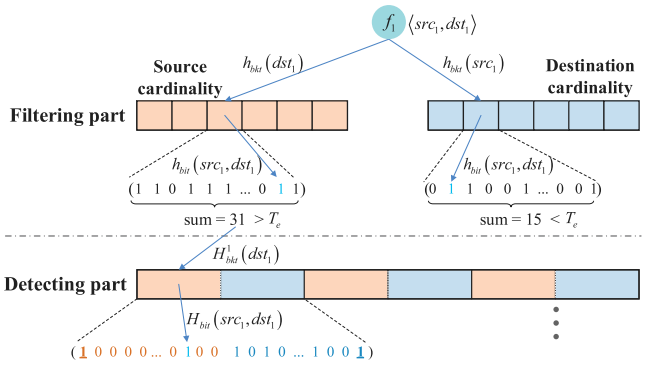


Fig. 3. An example of update operations.

corresponding hashed buckets. The abnormal flow with extremely high source cardinality (red), however, triggers distinct processing behavior. Its estimated source cardinality min88,81,86=86 just exceeds the T_{rcv} threshold of 80 after the update operation. This flow is recognized as an abnormal flow and its destination is identified as a super receiver, typically representing a DDoS attack victim. Following detection, UnivCar initiates the dedicated reconstruction operation to fully reconstruct the destination address.

This processing workflow corporated with the hierarchical data structure enables efficient resource allocation - preserving lightweight operations for common low-cardinality flows while deploying advanced detection mechanisms only for exceptional cases requiring deeper analysis. Furthermore, to reduce hash operations, we defined the pair of source and destination addresses (i.e., $\langle src, dst\rangle$) as the input flow and set it as the input key of the bitmap hash function. For the source cardinality estimation, we set the destination address (i.e., dst) as the key of the bucket hash function to determine the selected bucket. Similarly, we adjust the input key of the bucket hash function to the source address (i.e., src) in destination cardinality estimation.

3.3. Operations

UnivCar supports four operations: (1) update operation, which performs the insertion of each upcoming flow; (2) extension operation, which dynamically adjusts the sketch structure to fit the increasing host cardinalities; (3) query operation, which returns the estimated host cardinality of a given address; (4) reconstruction operation, which returns the address of a detected super host.

3.3.1. Update operations

At the beginning of each measurement epoch, all the bitmaps are initialized to 0, and $flag_src$ and $flag_dst$ are set to 1. To reduce computing complexity, the source cardinality and destination cardinality estimation share the same hash functions. The update operation in the filtering part is conducted with two hash functions: h_{bkt} is used for selecting bucket and h_{bit} is used for determining the bit position of each hashed bucket. Similarly, the detecting part of UnivCar possess d bucket-selection hash functions $H^i_{bkt}(i = 1, 2, \ldots, d)$ and one bit-locating hash function H_{bit}. To enable reconstruction of super host address, each bucket-selection hash function satisfies $H^i_{bkt}(x) \equiv x \pmod{W_i}$ based on the Chinese Remainder Theorem. For each incoming flow f with source–destination pair $\langle src, dst\rangle$, the update operations are detailed as follows.

Update operations in the filtering part. When estimating the source cardinality of flow f, UnivCar hashes the destination address to select a bucket $h_{bkt}(dst)\%w$. For each selected bucket, UnivCar hashes the source–destination pair $\langle src, dst\rangle$ and normalizes the hashed value $h_{bit}(src, dst)$ to be with range $[0, m-1]$ to determine the bit position. Then UnivCar checks the selected bit cell $(h_{bkt}(dst)\%w) * m + h_{bit}(src, dst)$ and set it to 1. Similarly, destination cardinality estimation follows the same process as the source cardinality estimation and updates the bit cell $(h_{bkt}(src)\%w) * m + h_{bit}(src, dst)$ to 1. The only difference between these two estimation tasks is that the input keys are not the same. We obtain the estimated result of the source/destination cardinality by computing the sum value of each bitmap (i.e., counting the number of 1s). Once the estimated value of the source cardinality or destination cardinality exceeds the predefined threshold T_e, the estimation task will be transferred to the detecting part.

Table 3
Description of different flag values.

flag_src	flag_dst	Description
1	1	estimate source and destination cardinalities simultaneously
1	0	only estimate source cardinality
0	1	only estimate destination cardinality
0	0	backup bucket for extension

Fig. 3 shows the update operations of an incoming flow f_1 with source–destination pair $\langle src_1, dst_1\rangle$. We set the threshold T_e as 30. In the filtering part, UnivCar hashes the source and destination address to select two buckets from source cardinality (SC) and destination cardinality (DC) bitmaps, respectively. Then UnivCar calculates $h_{bit}(src_1, dst_1)$ to get the mapped bits in two bitmaps and set them to 1 simultaneously. As the estimated value (i.e., number of 1s) of the selected bucket in the SC bitmap exceeds 30, UnivCar expels flow f_1 to the detecting part for further estimation. Notice that, even if the estimated value in the DC bitmap is less than T_e, UnivCar will estimate the destination cardinality of flow f_1 in the detecting part to preserve the consistency of different cardinality estimation tasks. In this case, UnivCar will flash the two selected buckets in the filtering part to provide memory space for the estimation of other flows.

Update operations in the detecting part. In source cardinality estimation, for each incoming flow with source–destination pair $\langle src, dst\rangle$, UnivCar hashes the source address with d pairwise-independent hash functions to select a bucket in each row. Destination cardinality estimation follows the same procedure but with a different input key (i.e., destination address). Each selected bucket is separated into two bitmaps: one is for source cardinality estimation, and the other one is for destination cardinality estimation. Besides, we introduce two additional flags (i.e., $flag_src$ and $flag_dst$) to characterize which kinds of cardinality estimation are being performed (detailed in Table 3). These two flags are set to 1 at the initial stage of each estimation epoch, representing that source cardinality and destination cardinality are being estimated simultaneously in each bucket.

When the source cardinality (or destination cardinality) measured value exceeds $\theta M (0 < \theta < 1)$, we set $flag_dst$ (or $flag_src$) to 0. The cardinality estimation in that bucket is transferred from sharing mode to exclusive mode, where only the source cardinality or destination

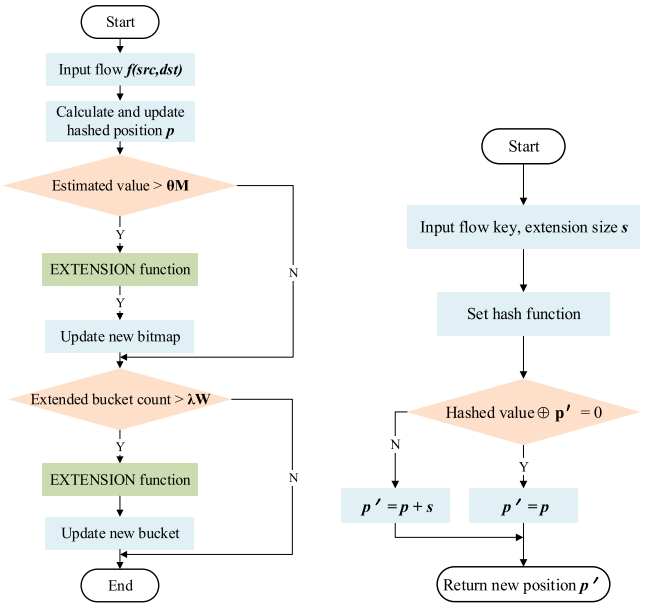


Fig. 4. The workflow of the extension algorithm (Algorithm 1).

cardinality is measured in the whole bucket. In this context, UnivCar avoids using two independent memory spaces to simultaneously conduct the source cardinality and destination cardinality measurement, reducing about half of the memory consumption. However, due to the existence of hash collisions, the cardinality information in the bitmap is inevitably missing and distorted during this transition, leading to inaccurate cardinality estimation.

3.3.2. Extension operations

To enable flexible and lossless transfer of host cardinality information, we propose a bitmap extension strategy based on Theorem 1. Besides, if too many extended bitmaps occur, which are correlated with super host accounts, we also need to conduct bucket extensions to ensure measurement accuracy. In this case, we present a bucket extension strategy to incorporate the available buckets for cardinality estimation. Algorithm 1 illustrates both strategies as they share the same mathematical foundation (i.e., Theorem 1), which introduces a novel modular arithmetic property when the modulus is odd. However, the proposed Theorem 1 becomes ineffective after the first extension where the modulus changes to even, thereby preventing consistent double-sized extensions. To address this problem, we further present Theorem 2 to generalize the modular extension capability of Theorem 1, and propose the general principle of double-sized extension.

THEOREM 1. *Given two arbitrary integers a and M, $a \bmod M = b$. When M is odd, if a and b have the same parities (i.e., odd or even), then $a \bmod 2M = b$. If a and b have different parities, then $a \bmod 2M = M + b$.*

Proof. Based on $a \bmod M = b$, we can derive that $a = kM + b$, where k is a natural number and $k = \frac{a-b}{M}$. Then we have $a \bmod 2M = (kM + b) \bmod 2M = (k \bmod 2)\, M + b$. When M is an odd number, k has the same parity as $(a - b)$. Besides, if a and b have the same parities, we have $(a - b) \bmod 2 = 0$. Otherwise, we have $(a - b) \bmod 2 = 1$. Therefore, Theorem 1 holds. □

Based on Theorem 1, we design Algorithm 1 to implement lossless extension operations in the detecting part. Without loss of generality, we take the super spreader detection as an example to illustrate the bitmap and bucket extension strategies in Algorithm 1. The corresponding workflow is depicted in Fig. 4. According to the update operations,

Algorithm 1 Extension operations of UnivCar

Input: Upcoming flow $f(src, dst)$, UnivCar sketch UC.
Output: Extended UnivCar with records updated UC'.

```
    #Take super spreader detection as an example
1:  for i = 1 to d do
2:      bkt = H^i_bkt(src) mod W_i
3:      bit = h_bit(⟨src, dst⟩) mod M
4:      UC[bkt][bit] = 1
5:      if A[src]_row_i > θM then
6:          update flag_src and flag_dst
7:          flash SC bitmap
8:          bit_new = Extension(⟨src, dst⟩, M)
9:          UC[bkt][bit_new] = 1
10:     end if
11:     count buckets with extended bitmap, cnt_row_i
12:     if cnt_row_i > λW_i then
13:         bkt_new = Extension(src, W_i)
14:         update new bucket UC[bkt_new]
15:     end if
16: end for
17: function Extension(key f, size s)
18:     if bitmap extension then
19:         set hash function H = h_bit
20:     else if bucket extension then
21:         set hash function H = H^i_bkt
22:     end if
23:     calculate original position p = H(f) mod s
24:     if H(f) ⊕ s = 0[1] then
25:         p' = p
26:     else if H(f) ⊕ s = 1 then
27:         p' = p + s
28:     end if
29:     return update position p'
30: end function
```

UnivCar records an upcoming flow $f(src, dst)$ by updating each hashed position to 1 (Line 2–4). When the estimated destination cardinality in a bucket exceeds the predefined threshold θM (Line 5), we perform bitmap extension by updating the $flag_src$ and $flag_dst$ to 0 and 1 respectively, and flashing SC bitmap to 0 (Line 6–7), indicating the selected bucket will only perform destination cardinality estimation for subsequent flows. Then we recall the EXTENSION function to calculate the position of newly selected bit after bitmap extension for updating (Line 8–9). Similarly, when the number of extended bitmaps exceeds the predefine threshold λW_i (Line 12), we perform the bucket extension by calculating the position of newly selected bucket and updating this bucket in terms of flags and destination cardinality (Line 13–14). Both the bitmap and bucket extension operations adopt the EXTENSION function (Line 17–30) which basically follows Theorem 1.

Bitmap extension in each bucket. When the cardinality estimation in a bucket transfers from sharing mode to exclusive mode, the bitmap possessed by one estimation task will be extended to the whole bucket (i.e., twice its initial size). For instance, if the network is under a scanning attack where destination cardinality estimation is more important, UnivCar will zero out all the bits of SC bitmap to release memory space for destination cardinality estimation. To achieve a lossless transferring of cardinality estimation in that process, we derive Theorem 1 for modulo arithmetic and present the bitmap extension strategy based on it. After performing bitmap extension in each bucket, UnivCar conducts the update operations of an incoming source–destination flow $\langle src, dst \rangle$ in two ways. If the key value has the same parity as its hashed bit position $h_{bit}(src, dst)\%M$, UnivCar sets this bit to 1. Otherwise, bit in the position $h_{bit}(src, dst)\%M + M$ will be set to 1.

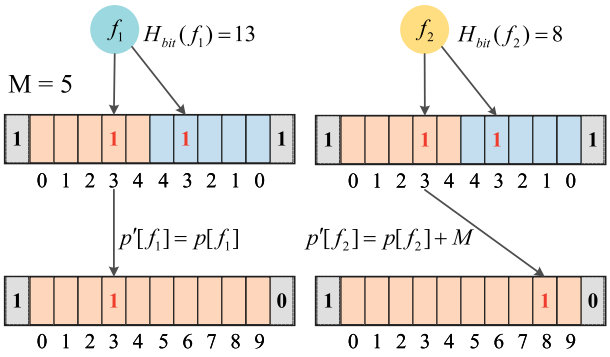


Fig. 5. An example of bitmap extension in each bucket.

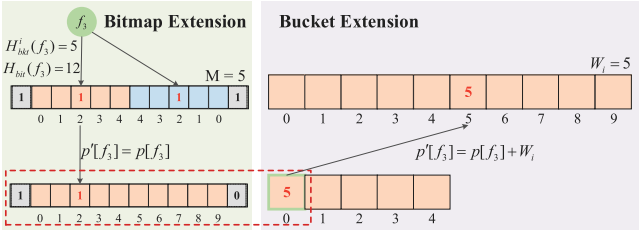


Fig. 6. An example of the entire extension operations.

Fig. 5 gives an example of the bitmap extension in each bucket. We set the length of the bitmap in each bucket as 5. Suppose f_1 and f_2 are two flows with high source cardinalities. They are mapped to the same bit position by the bit-locating hash function, i.e., $p[f_1] = H_{bit}(f_1) \bmod M = 13 \bmod 5 = 3$, $p[f_2] = H_{bit}(f_2) \bmod M = 8 \bmod 5 = 3$. After extension, UnivCar merges each bucket into one single bitmap and changes *flag_dst* to 0 to estimate source cardinality only. Since both the $H_{bit}(f_1)$ and $p[f_1]$ are odd numbers, the mapped position of flow f_1 after extension is equal to its original position (i.e., $p'[f_1] = p[f_1] = 3$). On the contrary, $H_{bit}(f_2)$ and $p[f_2]$ have different parities. Then we have $p'[f_2] = p[f_2] + M = 8$.

Bucket extension in each row. When the number of buckets transferred from sharing mode to exclusive mode exceeds threshold λW_i, we need to extend the size of the detecting part to prevent high hash function collision. Based on Theorem 1, UnivCar supports lossless and flexible double-sized extention of the detecting part when the bucket account of each row in the detecting part is an odd number. In general, the bucket extension is similar with bit extension while different in the input keys and hash functions (detailed in Algorithm 1).

Fig. 6 depicts an example of the entire extension operations in the detecting part. We set $M, W_i = 5$ and $\theta, \lambda = 0.8$. Suppose the upcoming flow f_3 is a high source-cardinality flow which is hashed to the No. 2 bit of the first bucket in the ith row, i.e., $H^i_{bkt}(f_3) \bmod W_i = 5 \bmod 5 = 0$ and $H_{bit}(f_3) \bmod M = 12 \bmod 5 = 2$. Similar with the flow f_1 in Example 2, the mapped bit position of flow f_3 is equal to its original position after bitmap extension, since the $H_{bit}(f_3)$ and $p[f_3]$ have the same parities. After the bitmap extension in the first bucket, the number of extended buckets in the ith row just exceeds the predefined threshold λW_i (i.e., $5 > 0.8 * 5$). This activates the bucket extension of the ith row. Since the $H^i_{bkt}(f_3)$ and $p[f_3]$ have different parities, the selected bucket position after bucket extension is $p'[f_3] = p[f_3] + W_i = 5$.

General principle of double-sized extension. Both the bitmap and bucket extension described above are based on the premise that the modulus in Theorem 1(i.e., bitmap length or bucket account) is an odd number. To enable the consistent double-sized extension, we further present Theorem 2 to ensure lossless transfer of host cardinality information during extension. When the modulus M becomes even after the first extension, Theorem 2 establishes a recursive mechanism by tracking the intermediate value k_n. This allows continuous extensions by checking the parity of the quotient term, effectively handling rapid traffic variations that trigger multiple extensions. Specifically, let M_n denotes the modulus at the nth expansion, then $M_{n+1} = 2M_n$. We can recursively deduce Theorem 2 by a multiple of 2 as follows.

Algorithm 2 Reconstruction operations of UnivCar

Input: super host $h(src, dst)$, UnivCar sketch UC.
Output: super host address.

```
#Take super spreader detection as an example
for i = 1 to d do
    H^i_bkt(srcIP) ≡ p_i (mod W_i)
    W = W * W_i
end for
for i = 1 to d do
    Q_i = W/W_i
    calculate Q_i^{-1} using Extended Euclidean Algorithm
    srcIP = srcIP + p_i * Q_i * Q_i^{-1}
end for
transfer srcIP (in decimal) to readable source address
return source address
```

THEOREM 2. *Given two arbitrary integers a and M_n, where n is a positive integer and $M_n = 2^{n-1}M$, we have $a \bmod M_n = b$, $k_n = \lfloor a/M_n \rfloor$. If k_n is an even number, then $a \bmod 2M_n = b$. If k_n is an odd number, then $a \bmod 2M_n = M_n + b$.*

Theorem 2 generalizes the modular extension capability of Theorem 1 from one time (i.e., $M \to 2M$), to n times (i.e., $M_n \to 2M_n$) by introducing an intermediate variable k_n. Besides, Theorem 1 can be viewed as a special instance of Theorem 2 (i.e., n = 1). In this context, we can continuously extend the detecting part based on Theorem 2 without reconfiguring the processing logic of the data stream when storage resources permit. However, considering the general extension requiring additional memory for recording k_n, it is unsuitable for deployment on some measurement devices with very limited memory, such as smartNICs [64]. Therefore, we only adopt the basic bitmap and bucket extension strategies in our work and perform the corresponding experimental evaluations in Section 6.

3.3.3. Query operations

The estimated results are mainly observed from the detecting part, which basically follows the estimation principle of a classical CM sketch [52]. To obtain the cardinality estimation results of each host, UnivCar first locates the buckets corresponding to that host in each row. Then, UnivCar checks the flag in each bucket to determine which cardinality measurement task is being performed. The source/destination cardinality of a host h corresponds to the minimum results among all d rows:

$$\mathcal{A}[h] = min\left\{\mathcal{A}[h]_{row_1}, \mathcal{A}[h]_{row_2}, \ldots, \mathcal{A}[h]_{row_d}\right\} \quad (1)$$

The estimated results of host cardinalities can be calculated based on Eq. (1). UnivCar compares the results with predefined thresholds to identify the super hosts as follows:

Super spreader estimation. A host whose destination cardinality exceeds the predefined threshold T_{spr} can be recognized as a super spreader.

Super receiver estimation. A host whose source cardinality exceeds the predefined threshold T_{rcv} can be recognized as a super receiver.

Super changer estimation. Since scanning attacks are usually the primary activities of an attack chain, we identify a host as a super changer when its increment of destination cardinality between two adjacent time intervals exceeds the predefined threshold T_{ch}.

3.3.4. Reconstruction operations

After identifying the super hosts, reconstruction operations of their addresses should be conducted to inform these abnormal hosts and prevent further damage to the network. UnivCar reversibly calculates super host addresses using the Chinese Reminder Theory (CRT). We take the reconstruction of a super spreader whose destination cardinality exceeds the threshold T_{spr} as an example for illustration.

Basic reconstruction. For the $\langle src, dst\rangle$ with high destination cardinality, UnivCar mapped it into d buckets of the detecting part by hashing its source address and normalizing the hashed value within $[0, W_i - 1]$. This process is formalized as $H^i_{bkt}(srcIP) \equiv p_i \pmod{W_i}$, where p_i is the position index of mapped bucket in the ith row ($i = 1, 2, \ldots, d$). Since each bucket-selection hash function satisfies $H^i_{bkt}(x) \equiv x \pmod{W_i}$, we have the congruence equations.

$$\begin{cases} srcIP \equiv p_1 \pmod{W_1} \\ srcIP \equiv p_2 \pmod{W_2} \\ \cdots \\ srcIP \equiv p_d \pmod{W_d} \end{cases} \tag{2}$$

Based on the CRT, we can derive the source address (in decimal) as $k\mathcal{W} + \sum_{i=1}^{d} p_i Q_i Q_i^{-1}$, where $\mathcal{W} = \prod_{i=1}^{d} p_i$, $Q_i Q_i^{-1} \equiv 1 \pmod{W_i}$, $Q_i = \mathcal{W}/W_i$ and k is a natural number. To ensure a unique solution (i.e., $k = 0$), $\mathcal{W}$ must be at least equal to the total number of all distinct source addresses.

Reconstruction with bucket extension. In particular, when the detecting part extends to double its original size, we can formalize the update process as $srcIP \equiv p'_i \pmod{2W_i}$. Based on the transferability of the congruence theorem, we can transfer each congruence equation as

$$\begin{cases} srcIP \equiv p'_i \pmod{W_i} \\ srcIP \equiv p'_i \pmod{2} \end{cases} \tag{3}$$

According to Theorem 1, we can reduce the congruence equations to Eq. (2) and calculate the unique solution based on CRT. Algorithm 2 illustrates the reconstruction process of a super host address.

4. Theoretical analysis

In this section, we present the theoretical analysis of our UnivCar sketch in terms of the estimation accuracy and complexity. Because the measurement process of UnivCar for the source cardinality and the destination cardinality is exactly the same except for the different input keys. Therefore, we take the estimation of source cardinality as an example to analyze the estimation error bound and complexity of UnivCar sketch.

4.1. Error bound

Let N denote the number of all distinct source IPs. We set d, W, and M as the parameters of UnivCar (see Table 2) and present the following theorems and their proofs. Our theoretical analysis is based on two key assumptions:

- *Uniform Hashing*: The hash functions distribute both source and destination IP addresses uniformly across buckets. This assumption is consistent with most prior sketch-based methods [2] and can be implemented using widely adopted algorithms such as Jenkins [65] and MurmurHash [66].
- *Stationary Measurement Window*: The total count of distinct IP addresses, N, remains relatively stable within a measurement epoch. As a sketch-based network monitoring solution, UnivCar employs a second-level measurement interval, which is significantly shorter than the sampling intervals used by many practical network measurement tools (e.g., NetFlow [19] and sFlow [20]). Even in dynamic networks where N may exhibit significant variation, our analysis still holds for each individual measurement epoch. Network operators can dynamically adjust the parameters W and M between epochs using the error bound relationships outlined in Remarks 1–4 to prevent rapid scaling of error bounds.

4.1.1. Error bound of each bucket

THEOREM 3. *When there are n_s source IPs mapped into a bucket, the estimated value $\hat{n}_s$ satisfies*

$$P\left\{|\hat{n}_s - n_s| < \varepsilon\left(N - M\left(1 - e^{-\frac{N}{M}}\right)\right)\right\} > 1 - \frac{1}{\varepsilon} \tag{4}$$

Proof. Since there are M bits for source cardinality estimation in each bucket, the probability of each bit X_i to be touched after processing one source IP can be expressed as $P(X_i = 1) = \frac{1}{M}$, then we have $P(X_i = 0) = 1 - \frac{1}{M}$. After processing n_s distinct source IPs, each bucket in the detecting part satisfies

$$P\{X_i = 1\} = 1 - P\{X_i = 0\} = 1 - \left(1 - \frac{1}{M}\right)^{n_s} \tag{5}$$

Since the detecting part only record high cardinality, M is large enough that we can reduct the above formula to $P\{X_i = 1\} = 1 - e^{-\frac{n_s}{M}}$. The expected estimated value of each bitmap satisfies

$$\begin{cases} E(\hat{n}_s) = \sum_{i=1}^{M} X_i P\{X_i = 1\} = M\left(1 - e^{-\frac{n_s}{M}}\right) \\ E(\hat{n}_s) \le n_s \end{cases} \tag{6}$$

Because source cardinality estimation records each distinct source address only one time, not every element increments the value of the bitmap. Thus, we have

$$E(n_s) - E(\hat{n}_s) = n_s - M\left(1 - e^{-\frac{n_s}{M}}\right) \le N - M\left(1 - e^{-\frac{N}{M}}\right) \tag{7}$$

According to the Markov inequality, we can derive that

$$\begin{aligned} &P\left\{|\hat{n}_s - n_s| \ge \varepsilon\left(N - M\left(1 - e^{-\frac{N}{M}}\right)\right)\right\} \\ &\le \frac{E(n_s) - E(\hat{n}_s)}{\varepsilon\left(N - M\left(1 - e^{-\frac{N}{M}}\right)\right)} \\ &\le \frac{N - M\left(1 - e^{-\frac{N}{M}}\right)}{\varepsilon\left(N - M\left(1 - e^{-\frac{N}{M}}\right)\right)} = \frac{1}{\varepsilon} \end{aligned} \tag{8}$$

Therefore, Theorem 3 holds. □

Remark 1. When $M \gg N$, we can calculate that the error range of Theorem 3 approximates $\varepsilon N^2/2M$ by substituting the Taylor expansion of $e^{-N/M}$; When $M \ll N$, the error range of Theorem 3 is close to $\varepsilon(N - M)$. Therefore, increasing the bitmap length (i.e., M) of the detecting part can reduce the estimation error of each bucket.

Remark 2. The confidence level of the estimation result in each bucket (i.e., the probability that the estimation error does not exceed a given range) is directly related to the parameter ε. The larger ε is, the higher the confidence level (i.e., $1 - 1/\varepsilon$) of the estimation result is, while the error range increases linearly with ε.

4.1.2. Error bound of UnivCar sketch

THEOREM 4. *Let C_s and $\hat{C}_s$ represent the real and estimated source cardinality respectively.*

$$P\left\{\hat{C}_s - C_s < \varepsilon M\left(1 - e^{-\frac{N}{MW}}\right)\right\} > 1 - \varepsilon^{-d} \tag{9}$$

Proof. Since hashed indexes generally adhere to uniform distribution, the expected number of hash collision is $\frac{N-C_s}{W}$ [52]. Thus, the actual expected number of source IPs mapped to the same bucket is $C_s+\frac{N-C_s}{W}$. Then we have

$$\begin{aligned} E\left(\hat{C}_s\right) &= \sum_{i=1}^{M} X_i P\left\{X_i = 1\right\} \\ &= M\left(1-e^{-\frac{C_s+\frac{N-C_s}{W}}{M}}\right) \\ &< M\left(1-e^{-\frac{WC_s+N}{MW}}\right) \end{aligned} \tag{10}$$

Based on (Eq. (6)), the estimated source cardinality in the ith bucket $\hat{C}_s^i$ satisfies

$$\begin{aligned} E\left(\hat{C}_s^i\right) - E\left(C_s\right) &\le E\left(\hat{C}_s^i\right) - M\left(1-e^{-\frac{C_s}{M}}\right) \\ &= M\left(1-e^{-\frac{WC_s+N}{MW}}\right) - M\left(1-e^{-\frac{C_s}{M}}\right) \\ &< M\left(1-e^{-\frac{N}{MW}}\right) \end{aligned} \tag{11}$$

According to the Markov inequality, we can derive that

$$\begin{aligned} & P\left\{\hat{C}_s^i - C_s \ge \varepsilon M\left(1-e^{-\frac{N}{MW}}\right)\right\} \\ & \le \frac{E\left(\hat{C}_s^i\right) - E\left(C_s\right)}{\varepsilon M\left(1-e^{-\frac{N}{MW}}\right)} \\ & \le \frac{M\left(1-e^{-\frac{N}{MW}}\right)}{\varepsilon M\left(1-e^{-\frac{N}{MW}}\right)} = \frac{1}{\varepsilon} \end{aligned} \tag{12}$$

Based on the strategy of a classical CM sketch, the estimated cardinality in the UnivCar sketch is the minimum value of d mapped buckets, i.e., $\hat{C}_s = \min\left\{\hat{C}_s^1, \hat{C}_s^2 \cdots, \hat{C}_s^d\right\}$. Then we have

$$\begin{aligned} & P\left\{\hat{C}_s - C_s \ge \varepsilon M\left(1-e^{-\frac{N}{MW}}\right)\right\} \\ & = \prod_{i=1}^{d} P\left\{\hat{C}_s^i - C_s \ge \varepsilon M\left(1-e^{-\frac{N}{MW}}\right)\right\} \le \varepsilon^{-d} \end{aligned} \tag{13}$$

Therefore, Theorem 4 holds. □

While our analysis assumes ideal hashing, real-world hash functions may exhibit χ^2-divergence δ from uniformity [67]. Through worst-case analysis, the collision term $\frac{N-C_s}{W}$ in Theorem 4 becomes $\frac{(1+\delta)(N-C_s)}{W}$, implying our bounds hold with ε scaled by $(1+\delta)$. As many modern hash functions can achieve trivial δ [65,66], this introduces negligible error inflation of the measurement results.

Remark 3. The detecting part of UnivCar adopts the CM sketch strategy which inherently mitigates transient overloads in dynamic environments. When a subset of buckets experiences sudden load spikes, CM sketch automatically discards outlier estimates from overloaded buckets. This structural property is further proofed in Theorem 4 that increasing the number of hash functions (i.e., depth d) can exponentially improve the confidence level (i.e., $1-\varepsilon^{-d}$) of measurement results. Empirical evidence suggests that $d \ge 3$ delivers sufficient accuracy for most CM sketch implementations [3,22,52], thereby reinforcing the robustness and effectiveness of this approach in handling dynamic and unpredictable data environments.

Remark 4. When configuring the data structure of UnivCar, the measurement load of the detecting part should be fully considered:

- *Case 1 (Low-load measurement: $N \ll MW$):* The estimation error range approximates $\varepsilon N/W$. This indicates that increasing/decreasing the bitmap length (i.e., M), has no significant effect on the error range, while increasing the number of buckets in each row (i.e., W), decreases the error range.
- *Case 2 (High-load measurement: $N \gg MW$):* The estimation error range approaches εM. Though reducing M may lower error range theoretically, frequent hash collisions make such measurements unreliable. Optimal strategy involves transferring to the low-load measurement case through the coordinated scaling of M and W.

Therefore, when configuring the data structure of UnivCar, the primary requirement is to assign M a sufficiently large value to prevent the system from operating in the high-load measurement case characterized by frequent hash collisions. Crucially, W should be prioritized for increase to minimize the measurement error within given resource constraints.

4.1.3. Worst-case analysis

To clarify the performance limits of UnivCar, this section presents the worst-case error bounds analysis in addition to the average-case analysis in Theorems 3 and 4.

Corollary 1. *When all N source IPs are mapped to a single bucket ($n_s = N$), the estimation error satisfies:*

$$\left|\hat{n}_s - N\right| = N - M\left(1-e^{-N/M}\right) \ge N - M \tag{14}$$

with equality when all bits in the M-length bitmap are set to 1.

Proof. Based on Eq. (6) in Theorem 3, the expected estimated source cardinality becomes:

$$E(\hat{n}_s) = M\left(1-e^{-N/M}\right) \tag{15}$$

Then we have the absolute estimation error as:

$$n_s - E(\hat{n}_s) = N - M\left(1-e^{-N/M}\right) \ge N - M \tag{16}$$

Since the maximum estimated value of each bucket is M (i.e., all bits are updated to 1), the minimum estimation error becomes $N-M$. □

This case can be further extended to the whole detecting part with d rows: N source addresses are mapped into only one bucket in each row, and the lower bound of the measurement error is also $N-M$. However, this worst case is far away with most general cases, losing practical meaning. Thus, we present the following worst-case analysis on the error bound of UnivCar sketch.

Corollary 2. *For the UnivCar sketch with depth d and width W, the worst-case estimation error satisfies:*

$$\hat{C}_s - C_s \ge C_s - M\left(1-e^{-C_s/M}\right) + \Delta_{min} \ge N - M \tag{17}$$

where $\Delta_{min} = M\left(1-e^{-\frac{N-C_s}{MW}}\right)$, denotes the minimum error from d independent estimations.

Proof. In the worst case, UnivCar sketch hashes all N source IPs to a single bucket in each row. Based on Corollary 1, when all bits of the mapped d buckets are set to 1, the error is lower bounded by $N-M$ with exponentially small probability W^{-d}, since $\hat{C}_s = \min\left\{\hat{C}_s^1, \hat{C}_s^2 \cdots, \hat{C}_s^d\right\}$.

Based on Theorem 4, we consider a more practical worst-case: among d rows, there exists at least one row that all N source IPs are mapped to a single bucket, while the expected number of source IPs of the mapped bucket in other rows is $C_s + \frac{N-C_s}{W}$. Then we have

$$\begin{aligned} C_s - E(\hat{C}_s) &= \min_{1\le i\le d}\left\{C_s^i - M\left(1-e^{-\frac{C_s^i+\frac{N-C_s^i}{W}}{M}}\right)\right\} \\ &\ge C_s - M\left(1-e^{-C_s/M}\right) + \underbrace{M\left(1-e^{-\frac{N-C_s}{MW}}\right)}_{\Delta_{min}} \end{aligned} \tag{18}$$

Table 4
Time and memory complexity comparison.

	Memory	Update time	Query time	Reconstruction time
VBF [24]	$O(2wk \log\log \lvert E\rvert)$	$O(2 \log\log \lvert E\rvert)$	$O(w \log\log \lvert E\rvert)$	$O(n^{\log\log \lvert E\rvert})$
DCDS [50]	$O(2dwk)$	$O(2d)$	$O(wd)$	$O(n^d)$
SpreadSketch [5]	$O(2k + 2\log \lvert E\rvert + 2dw \log\log \lvert E\rvert)$	$O(2d)$	$O(wd)$	$O(n)$
INDDoS [3]	$O(2dwk)$	$O(2d)$	$O(nd)$	–
SuperSketch [8]	$O(dwk^2)$	$O(3d)$	$O(dwk + \log d)$	$O(nd)$
UnivCar	$O(dwk)$	$O(2d)$	$O(nd)$	$O(nd \log w)$

The first term $C_s - M\left(1 - e^{-C_s/M}\right)$ represents the estimated error of C_s source IPs, and Δ_{min} denotes the expected irreducible error from the CM sketch strategy. Notably, when all N source IPs are mapped to a single bucket in all d rows (i.e., $C_s^1, C_s^2 \cdots, C_s^d = N$), the estimation error is also lower bounded by $N - M$. □

4.2. Memory and computing complexity

We make a complexity comparison between the proposed UnivCar sketch and several state-of-the-art sketches that are designed for super host identification, including VBF [24], DCDS [50], INDDoS [3], SpreadSketch [5] and SuperSketch [8]. To guarantee the fairness of the comparison, each sketch uniformly consists of d rows with w buckets in each row. Besides, we set the counter size of each bucket as k, which equals to the bucket size of the detecting part in UnivCar. Assume all the distinct counters and bit arrays used in each sketch have $O(1)$ time complexities, and there are n super hosts in each measurement epoch.

Since only SuperSketch [8] and UnivCar support estimating source and destination cardinality simultaneously, we set the memory space and update time of other sketches as twice of their basic value for comparison fairness. When the domain size of IP addresses is $|E|$, the memory and time complexities of these sketches are presented in Table 4.

Memory space. In UnivCar, the filtering part consists of large amounts but small-sized bitmaps in one row, occupying much smaller memory space than the detecting part. Thus, the memory complexity of UnivCar is mainly determined by the detecting part. In general, the memory complexity of UnivCar is $O(dwk)$, preserving the smallest memory consumption among these sketches.

Update time. In UnivCar, each flow with a small cardinality is estimated in the filtering part, requiring 3 hash operations and 2 bucket accesses. For each flow with high cardinality, the detecting part of UnivCar performs $2d + 1$ hash operations and $2d$ bucket accesses. Therefore, the per-flow update time complexity of UnivCar is $O(2d)$.

Query time. For super host identification, UnivCar only conducts the query operations in the detecting part. According to Eq. (1), UnivCar requires d bucket accesses for each super host to obtain its source or destination cardinality. Thus, the query time complexity of UnivCar is $O(nd)$, achieving the smallest value among all these sketches since n is much smaller than w.

Reconstruction time. UnivCar reconstructs the address of each super host using the Chinese Reminder Theorem (CRT), which relies on the Extended Euclidean Algorithm to sequentially compute d modular inverses with $O(d \log w)$ time complexity. Therefore, the reconstruction time complexity of all super hosts in UnivCar sketch is $O(nd \log w)$, which is much smaller than VBF [24] and DCDS [50]. Although SpreadSketch [5] achieves the lowest reconstruction time complexity (i.e., $O(n)$), it requires additional memory to record the address of each super host. The reconstruction time complexity of UnivCar is close to SuperSketch [8], firmly proofed by the experimental results in Section 6.4.

5. Implementation

5.1. Dual-mode hierarchical architecture with elastic scaling

UnivCar employs a hierarchical and extensible architecture to enable efficient cardinality estimation of skewed and dynamic traffic in large-scale network. It adaptively switches estimation modes and ensures high performance via passive migration during extension process.

5.1.1. Core architecture for efficient cardinality estimation

Hierarchical Data Structure. The UnivCar sketch employs a hierarchical architecture for efficient super host detection. The filtering part performs preliminary filtering through lightweight host cardinality estimation. It effectively filters out benign flows with estimated cardinalities below T_e. The detecting part implements dynamic dual-mode estimation to address complex scenarios. A flags array enables runtime switching between sharing and exclusive modes: maintaining memory-efficient sharing mode under normal conditions while automatically activating precision-oriented exclusive mode during high-cardinality traffic surges. In our C++ implementation, each bitmap is implemented by a series of 32-bit integers, and the front two bits are set as flags to indicate the current estimation mode (sharing/exclusive) and what kind of cardinality is being estimated (detailed in Table 3).

Integration of the filtering part and the detecting part. To enhance the UnivCar sketch's capability to rapidly classify and process traffic with varying cardinalities, our implementation introduces an additional small Bloom filter to record high-cardinality flows that have been evicted from the filtering part. Taking super spreader detection as an example, when a flow $f(src, dst)$ arrives, UnivCar first checks whether the source address (i.e., src) has been hashed into the Bloom filter. If present, this indicates the flow has previously been expelled from the filtering part and will consequently undergo continuous estimation in the detection part. If absent, the flow is mapped to the filtering part for cardinality estimation. As a proven probabilistic data structure, the Bloom filter achieves efficient set membership queries with minimal memory and computational overhead [68], while strictly guaranteeing zero false negatives: If all hash results indicate zero, the item is definitively absent. This inherent property naturally fulfills the requirement of UnivCar for fast traffic monitoring across different cardinality scales in large-scale network scenarios.

5.1.2. Elastic scaling mechanisms for rapid traffic variations

Rapid variations of high-cardinality traffic can trigger two scaling mechanisms in UnivCar: bitmap extension in each bucket and bucket extension in each row. In this context, the lossless extension algorithm avoids proactive bit migration during extension processes, as such immediate full migration would incur substantial latency penalties and degrade update throughput. Instead, UnivCar implements a passive migration approach where flow redirection occurs only upon the arrival of subsequent packets belonging to affected flows. When a packet from a flow mapped to the extended bitmap/bucket arrives, the lossless extension algorithm dynamically calculates its new position based on the Algorithm 1. By deferring migration until actual flow activity resumes, this relaxed update mechanism achieves elastic scaling while preserving consistent update performance throughout traffic variation cycles.

5.2. Selection criteria of hash functions

In general, h_{bit}, h_{bkt} and H_{bit} are designed based on the Jenkins hash function [65] to achieve a trade-off between processing speed and collision rate, while H^i_{bkt} satisfying $H^i_{bkt}(x) \equiv x(\mathrm{mod}W_i)$ to enable reconstruction of each super host address based on the Chinese Remainder Theorem (CRT).

5.2.1. Hash functions for large-scale network monitoring

In large-scale network traffic monitoring systems, hash function design must balance two critical requirements: (a) computational efficiency, to handle high-speed network traffic in real-time, and (b) output uniformity, ensuring hash values distribute randomly across the output space to minimize collision rates. In general, more complex hash functions can produce more uniform outputs but also require more computation time. The specific design of the hash function represents a trade-off between complexity and ease of implementation.

We employ three variant Jenkins-based hash functions (h_{bit}, H_{bit} and h_{bkt}) that optimize this balance through XOR/shift operations [69]. As a non-cryptographic hash algorithm, the Jenkins hash function is characterized by high computational efficiency and low collision rates. Cheng et al. [70] further prove that XOR operations inherently enhance distribution uniformity, making such constructs particularly suitable for our requirements. In our implementation, we fixed two seeds as the golden section constant (i.e., 0x9e3779b9) to enhance the avalanche effect of Jenkins hash function, and dynamically select a random big prime as the third seed to generate different hash functions.

5.2.2. Hash functions for reconstruction operations

To realize the reconstruction of super host addresses, we design a set of hash functions $H^i_{bkt}(i = 1, 2, \ldots, d)$ based on the CRT. Each hash function satisfies $H^i_{bkt}(x) \equiv x(\mathrm{mod}W_i)$, where x represents the host address to be hashed (source or destination IP), W_i is the modulus of the mutual prime (i.e., the number of buckets in ith row of the detecting part). In particular, when the detecting part extends to double its original size, the corresponding hash function will also be changed to $H^i_{bkt}(x) \equiv x(\mathrm{mod}2W_i)$ to avoid high collision rates. As illustrated in Section 3.3.4, we can derive each super host address (in decimal) based on the CRT both before and after performing the extension operations. Algorithm 2 illustrates the reconstruction process of a super host address.

6. Experiment and evaluation

In this section, we evaluate the performance and accuracy of UnivCar sketch based on the real-world traffic traces. We develop UnivCar sketch in C++ and conduct all the experiments on the LinuxOS (Ubuntu 20.04) with an Intel(R) Xeon(R) Gold 5218 CPU@2.30 GHz and 256 GB RAM. Furthermore, we flash the detecting part of UnivCar sketch with two different strategies: one is based on a fixed time interval (5s in our experiments), and the other one is conducted once the super host is identified. We recognize the latter strategy as UnivCar-flash in our evaluation results.

6.1. Experiment setup

Dataset. We perform the experiments by estimating the practical dataset CAIDA [71], which contains one-hour traffic traces captured on 10 GigE backbone links in the Internet. We set the estimation time interval as $5s$, resulting in 720 subtraces in each experiment. Each subtrace contains $\sim 2.3M$ packets with $\sim 60K$ distinct source addresses and $\sim 66K$ distinct destination addresses. The general cardinality distribution is illustrated in Fig. 7. We observe that both the source and destination cardinalities in the traffic traces are high skewed. For example, the sources (or destinations) with destination (or source) cardinality less than 10 account for over 97% of all distinct flows. Based on the thorough analysis of the cardinality distribution, we dynamically adjust the thresholds to keep the number of true super receivers/super spreaders/super changers in each subtrace as 50.

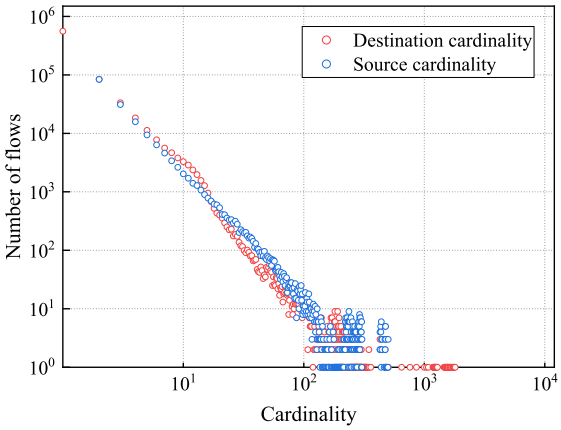


Fig. 7. The distribution of host cardinalities in a subtrace.

Configurations. We conduct a series of comparative experiments with the state-of-the-art approaches including INDDoS [3], SpreadSketch [5] and SuperSketch [8]. In each experiment, we fix the detecting part depth of UnivCar sketch as 3 and vary the number of buckets and bitmap width in each bucket. To guarantee the fairness of the comparison experiments, we tune the data structure configuration of other approaches under the same memory size and choose the setting that maximizes their performance and accuracy. Notice that, only SuperSketch and our proposed method support multiple tasks of super hosts identification simutaneously. INDDoS aims to detect DDoS victims(i.e., super receivers) while SpreadSketch focus on super spreader identification. We configure them with different input keys to support both super receiver and spreader detection. Thus, we only make comparison with SuperSketch for the experiment results of super changer identification.

Metrics. Given H_{all} represents the set of all the input flows. Let H_{est}, H_{true} denote the set of estimated and real super hosts, respectively. Then we have TP(true positive) = $|H_{est} \cap H_{true}|$, FP(false positive) = $|H_{est}| - TP$, and FN(false negative) = $|H_{true}| - TP$. Based on these basic metrics, we choose the following metrics for evaluation:

(1) Recall: $\frac{TP}{|H_{true}|}$, the ratio of true super hosts detected over all true super hosts;
(2) Precision: $\frac{TP}{|H_{est}|}$, the ratio of true super hosts detected over all detected super hosts;
(3) F1 score: $\frac{2\times RR\times PR}{RR+PR}$, the harmonic average of precision and recall;
(4) FPR(False Positive Rate): $\frac{FP}{|H_{all}-H_{true}|}$, the ratio of normal hosts estimated as super hosts;
(5) FNR(False Negative Rate): $\frac{FN}{|H_{true}|}$, the ratio of super hosts estimated as normal hosts;
(6) RE(Relative Error): $\frac{1}{n}\sum_{i=1}^{n}\frac{|f_i-\hat{f}_i|}{f_i}$, where n is the number of true super hosts detected, and f_i and $\hat{f}_i$ are the actual and estimated cardinalities, respectively.
(7) Throughput: the number of packets processed per second.
(8) Time consumption: the time for super host identification and address reconstruction.

6.2. Accuracy evaluation of cardinality estimation

Since high host cardinality is usually correlated with a super host, we first evaluate the estimation accuracy of different approaches regarding source cardinality, destination cardinality, and destination cardinality change. Note that the relative error of all estimated flows is

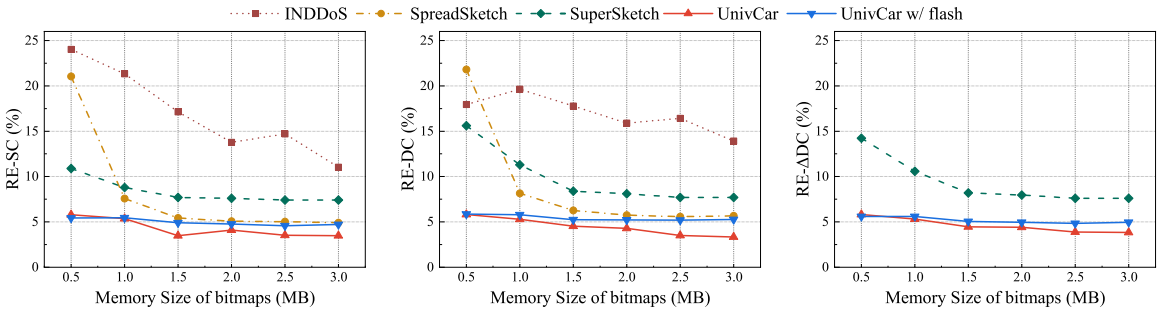


Fig. 8. The experiment results of cardinality estimation. SC: Source Cardinality; DC: Destination Cardinality; ΔDC: destination cardinality change.

meaningless for identifying super hosts. Thus, we focus on the relative error of potential super hosts with high host cardinalities in each experiment. Fig. 8 shows the experiment results of each estimation approach with different memory size. In general, our methods perform the lowest estimation error, and INDDoS has the worst cardinality estimation accuracy. When the memory size exceeds 1MB, the estimation errors of SpreadSketch referring to source and destination cardinalities are close to our methods. For the estimation of destination cardinality change, which is related to super changer identification, our methods outperform SuperSketch in varied size of memory.

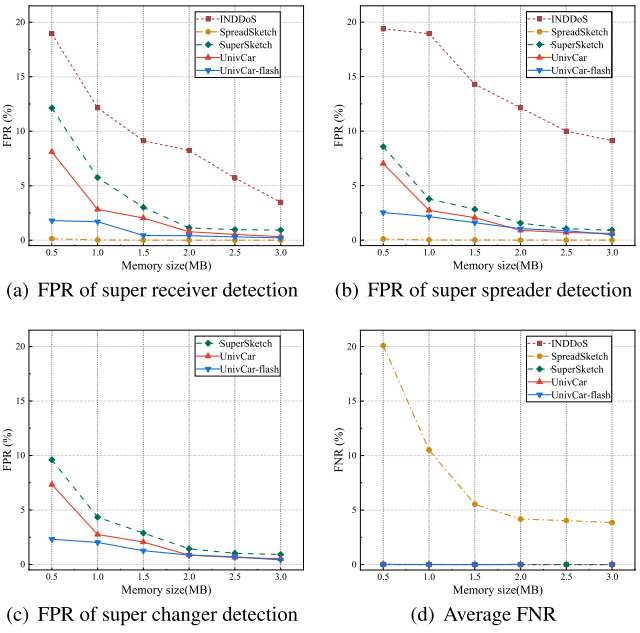
Both UnivCar and UnivCar-flash achieve no more than 6% relative error among different cardinality estimation tasks, lower than other approaches with different memory size. We also observe that UnivCar-flash has higher relative error than UnivCar. This is because the flash strategy makes the estimation value approach the cardinality threshold, which can be much lower than the cardinality of many heavy-tailed super hosts. Moreover, this also prevents the estimation error of UnivCar-flash from decreasing as memory increases, as UnivCar does.

6.3. Accuracy evaluation of super host identification

In this section, we evaluate the identification accuracy of UnivCar sketch for super hosts, including super receivers, super spreaders and super changers. For super changer identification, we only perform comparative experiments with SuperSketch since other approaches lack of this capability. The experiment results considering different memory size are illustrated in Figs. 9 and 10. In general, the detection accuracy of each approach is increased with the memory size(varied from 0.5MB to 3MB). Compared with previous work, UnivCar achieves higher accuracy in super host identification, especially when the memory size is less than 2MB.

Fig. 9 presents the FPR/FNR of each sketch for identifying different types of super hosts with varied memory sizes. Since the FNRs of all the evaluated sketches except SpreadSketch are close to 0, we only present the average results of the super receiver and spreader detection in Fig. 9(d) for clarity. Overall, UnivCar-flash achieves the best performance regarding the FPR and FNR evaluation results. When the memory size is no less than 1MB, both UnivCar and UnivCar-flash achieve no more than 3% FPR. SpreadSketch presents the lowest FPR but the highest FNR in the experiments with different memory sizes. This is because it uses additional memory space to track only one super host candidate, increasing the likelihood of missing other potential super hosts when the memory is limited. When the memory size is 0.5MB, UnivCar and SuperSketch show over 7% FPR, while UnivCar-flash presents about 2.5% FPR according to the experimental results in Fig. 9(a)~9(c). This benefits from the timely and efficient memory management of the flash strategy, addressing frequent hash collisions derived from small memory.

As illustrated in Fig. 10, SpreadSketch encounters greatly accuracy decrease when the memory size is 0.5MB. Its precision of super receiver detection (Fig. 10(a)) and the Recall of super spreader detection (Fig. 10(b)) are both lower than 46%. SuperSketch has the highest Recall among all the sketches, yet achieves much lower precision than our methods when the memory is no more than 1MB. The reason is that it acquires large basic memory to construct its multi-dimensional data structure. INDDoS allocates different hosts, skewd in cardinality distribution, with the size of bitmap which has to be large-scale to preserve estimation accuracy. When the memory size is less than 1.5MB, it presents an acceptable level of detection recall but achieve the worst detection precision. Due to the capability of the filtering part in filtering massive flows with low cardinalities and the extensible structure of the detecting part, UnivCar presents the highest F1 score among all sketches. For super changer identification, we observed that SuperSketch is prone to mistakenly identifying benign traffic as super host flows. Although it achieves nearly 100% detection recall, its detection precision is no more than 94%. When the memory size is set to 0.5MB, its detection precision and F1 score significantly drops to 67.6% and 80.2%, respectively. In contrast, the detection precision and F1 score of our methods are no less than 90% and 91%, respectively. Besides, UnivCar-flash achieves slightly higher detection accuracy than UnivCar in super changer identification.



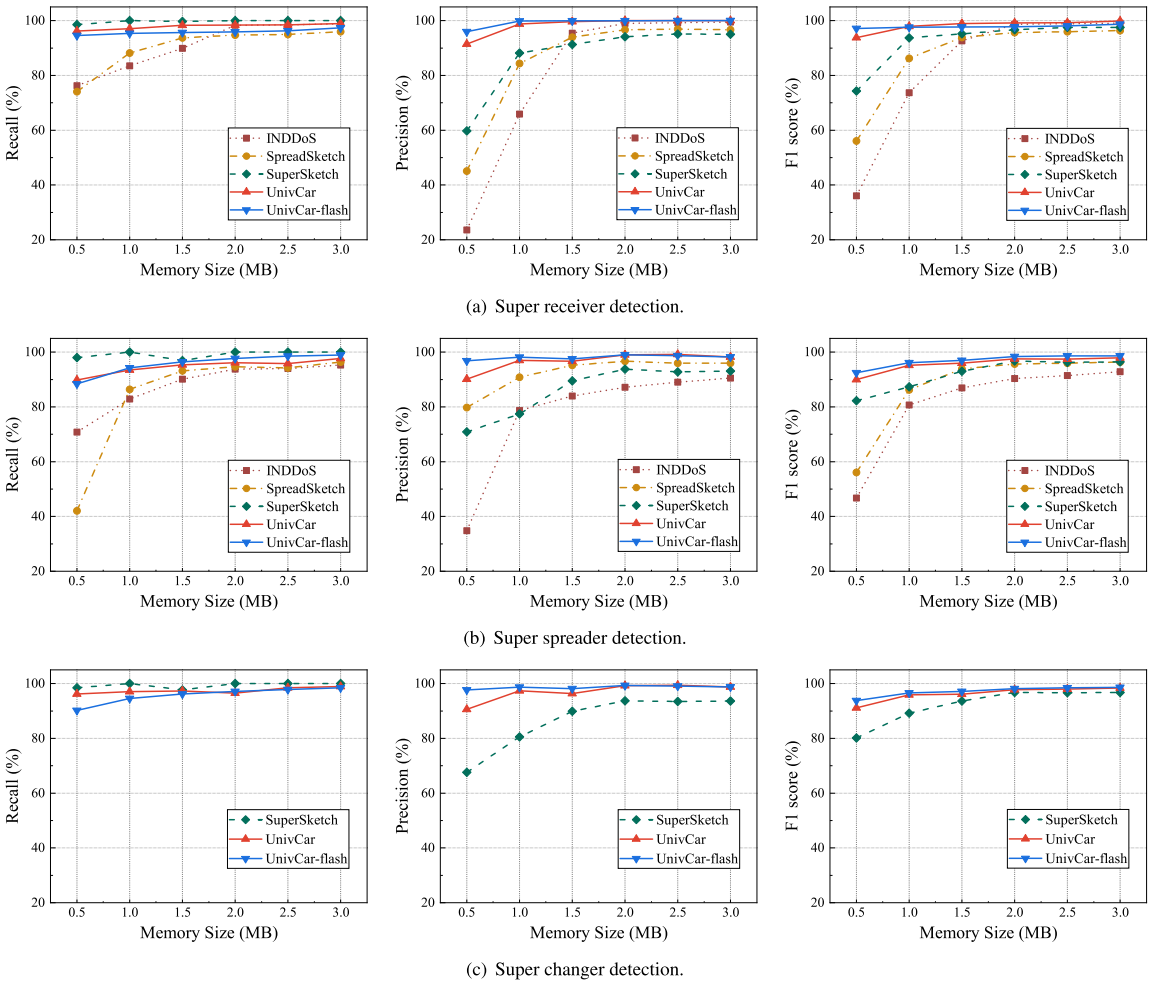
(a) FPR of super receiver detection (b) FPR of super spreader detection

(c) FPR of super changer detection (d) Average FNR

Fig. 9. FPR and FNR of super host identification with different memory size.

6.4. Performance evaluation

To guarantee the comparing fairness, we execute all approaches on the same platform and exclude the time consumption for loading each



(a) Super receiver detection.

(b) Super spreader detection.

(c) Super changer detection.

Fig. 10. Recall, precision, and F1 score of super host identification with different memory size.

subtrace. According to the experiment results in Figs. 8 and 10, we fix the memory size of all sketches as 1.5MB to preserve high performance. We take the average of multiple experiments as the final results and add error bars to each plot to show the differences.

Fig. 11 shows the update throughput of inserting the source–destination pairs of each subtrace into a sketch(in million packets per second). SpreadSketch achieves the highest update throughout as it acquires an efficient filtering method to handle massive flows with low cardinalities. Both UnivCar and UnivCar-flash achieve about 20Mpps update throughout and outperform INDDoS and SuperSketch due to the filtering capability of the filtering part. In particular, SuperSketch has the lowest throughout as it requires more operations to map a flow into the multiple dimensional sketch.

The time consumption results of update operation and address reconstruction are illustrated in Fig. 12. In general, UnivCar and UnivCar-flash achieve both high update and reconstruction performance. Fig. 12(a) shows that SuperSketch has the smallest update time among all the sketches since it has the least update operations. Our methods are about 1.6 and 3.1 times faster than INDDoS and SuperSketch, respectively. For the time consumption of reconstructing super hosts depicted in Fig. 12(b), SpreadSketch takes several milliseconds, while our methods and SuperSketch take tens of milliseconds. The reason is that SpreadSketch can export current super spreader directly by recording it in each bucket while consuming additional memory. Besides, this difference is trivial in practice compared with the time interval of each measuring epoch (i.e., 5s in our experiments). Notice that,

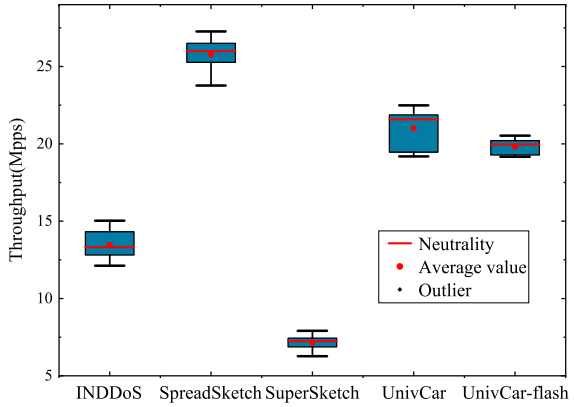
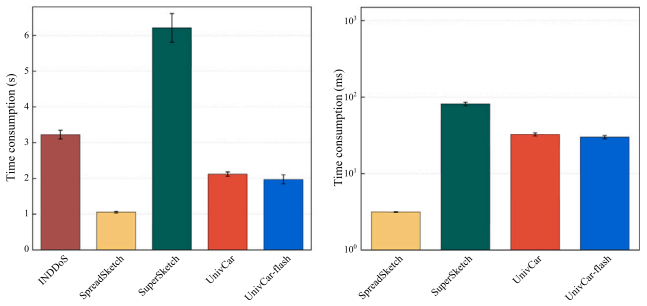


Fig. 11. The experiment results of update throughput.

since INDDoS does not support reconstruction, we did not design a comparison experiment.

6.5. Parameter sensitivity analysis

In this section, we perform comprehensive experiments regarding the effect of threshold values on super host detection accuracy. In each



(a) Update time (b) Address reconstruction time

Fig. 12. The experiment results of time consumption.

experiment, we take the average results of super spreader and super receiver detection as the final results.

6.5.1. Sensitivity to the data structure configuration

To investigate how the UnivCar's data structure affects the super host detection accuracy, we conduct a comprehensive sensitivity analysis regarding three key parameters: hash function quantity (d), bucket quantity (W), and bitmap size (M). We configure each row with the same number of buckets to better visualize the experimental results. The experimental results are presented in Fig. 13 with varied data structure configurations.

Hash Function Quantity (d): When increasing d from 1 to 5 in Fig. 13(a), the detection Precision and F1 score demonstrate rapid improvement while the detection Recall remains at a high level across all configurations. For instance, the F1 score ascends from 36.6% (d=1) to 99.3% (d=5), with the most significant gains (about 60% absolute improvement) occurring between d=1 and d=3. This phenomenon aligns with the theoretical expectation of Theorem 4 that improving the hash function quantity (i.e., the number of rows) can significantly reduce the collision probability.

Bucket Quantity (W): As shown in Fig. 13(b), the detection Precision and F1 score are very sensitive to the bucket quantity. As the W increases from 61 to 1021, they present significant growth: 3.8%–98.2% for Precision and 7.3%–99.1% for F1 score, respectively. This suggests that adequate bucket allocation can greatly reduce the detection false positives. Besides, when W reaches 509, UnivCar has been able to achieve a high detection accuracy (e.g., F1 score = 98.1%) and diminishing returns beyond this point.

Bitmap Size (M): As the bitmap size decreases to $\leq$ 255 bits in Fig. 13(c), all the metrics encounter great reduction, indicating that the bitmap size directly governs information capacity. Moreover, the Precision curve shows steeper growth compared to Recall and F1 score. Notably, practical implementation requires balancing memory efficiency, as the 4095-bit configuration provides only 1.1% absolute gain of F1 score over 2047-bit, but doubles the storage cost of each bitmap.

6.5.2. Sensitivity to thresholds configuration

In this section, we evaluate the impact of two critical thresholds on the update throughout and the detection accuracy of UnivCar: (1) T_e, the cardinality threshold for expelling flows from the filtering part to the detecting part; (2) θ, the extension threshold for switching the cardinality estimation from the sharing mode to the exclusive mode.

The impact of threshold T_e on the update throughout. We analyze the sensitivity of the update throughput to variations in the threshold T_e, which governs the transition of the high-cardinality flow from the filtering part to the detecting part of UnivCar sketch. The results are presented in Fig. 14, which illustrates the relationship between throughput and the threshold T_e which is presented as the percentage of all distinct source/destination IP addresses. As T_e increases from 0.25‰ to 1.50‰ of all distinct source/destination IPs, there is a noticeable increase in the update throughput of UnivCar. Even with a low threshold like 0.50‰, the update throughput of UnivCar remains about 19 Mpps. This performance improvement stems from dynamic workload redistribution between the filtering part and the detecting part. As the cardinality threshold increases, the filtering part progressively handles more and more arriving flows, reducing computational overhead by minimizing time-intensive update operations in the detecting part. However, we cannot continuously increase the threshold to enhance update throughput, as this would result in a growing number of high-cardinality flows being processed by the filtering part, which decreases the recall of super host detection in UnivCar.

The impact of threshold θ on the detection accuracy. Fig. 15 depicts the influence of threshold θ variation on detection accuracy under two memory configurations (0.5 MB and 1 MB). We adjust the memory size by varying the number of buckets, while the bitmap size under the two memory configurations is set to be the same to keep the experimental results comparable. Each subplot displays two metrics: F1 score (yellow bars) and FPR (green bars), evaluated across thresholds ranging from 0 to 0.20. Notably, when θ exceeds 0.2, the value of θM is greater than all the estimated cardinalities, and the experimental results are almost unchanged. Therefore, these results are not presented for clarity.

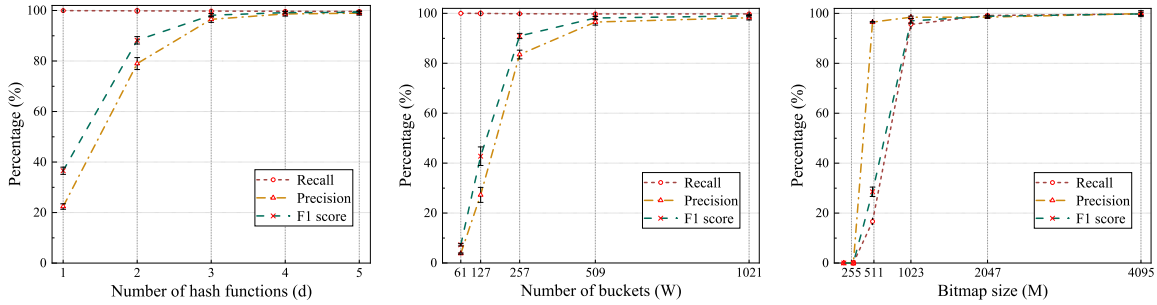
For the F1 score (yellow bars), both memory sizes exhibit high performance ($\geq$92.5%) across all thresholds. The values remain stable with small fluctuations (error bars show narrow ranges), suggesting that the detection accuracy is less sensitive to threshold adjustments. When θ varies from 0.01 to 0.12, the F1 score slightly declines but remains above 90% for both configurations. On the contrary, the error bars for FPR are slightly wider than those for F1 score, reflecting higher variability in false alarm rates. Compared with the results in Fig. 15(a), UnivCar with 1MB memory achieves a lower detection FPR ($\leq$ 9%). Moreover, we also observe that both F1 score and FPR encounter the lowest performance at a specific threshold (i.e., θ = 0.07), which is close to about 40% of the threshold for super host identification. This indicates the potential optimization opportunities through adaptive threshold selection.

6.6. Ablation study

We conduct a comprehensive ablation study on UnivCar to further determine the impact of each major component and extension strategy on the overall performance. Table 5 reports the experimental results regarding the super host detection accuracy of UnivCar and its variants with different memory sizes (i.e., varied from 0.5 MB to 3.0 MB). Specifically, we compare the full UnivCar sketch with the following variants:

- Without the filtering part (i.e., w/o FP). This variant removes the filtering part of UnivCar. It aims to evaluate the contributions of the filtering part on reducing the estimation overhead of the detecting part and improving the detection accuracy.
- Without the lossless extension strategy (i.e., w/o LE). This variant removes the lossless extension algorithm of UnivCar. It aims to evaluate the capability and efficiency of the proposed lossless extension strategy for bitmp and bucket extensions.

Impact of the Filtering Part. The absence of the filtering part leads to significant performance degradation, particularly when the memory size is lower than 1.5 MB. This highlights the critical role of the filtering part in tracking and filtering tremendous low-cardinality flows. It effectively filters out benign flows with estimated cardinalities below T_e, thereby reducing the estimation overhead of the detecting part and improving the precision of detecting super hosts with high cardinalities. For instance, the FPR respectively increases by 11.69% and 19.96% for UnivCar and UnivCar-flash at 0.5 MB, demonstrating its effectiveness in suppressing false alarms through preliminary feature selection.



(a) Sensitivity analysis to d (b) Sensitivity analysis to W (c) Sensitivity analysis to M

Fig. 13. Sensitivity analysis of the data structure configuration.

Table 5
Experimental results of the ablation study.

Memory Size	w/o FP			w/o LE			UnivCar			UnivCar-flash		
	F1	FPR	RE	F1	FPR	RE	F1	FPR	RE	F1	FPR	RE
0.5 MB	41.36	19.17	20.99	79.76	6.23	9.57	91.61	7.48	5.81	94.45	2.21	5.63
1.0 MB	77.19	15.55	20.49	91.79	4.46	8.23	96.32	2.77	5.32	96.79	1.96	5.61
1.5 MB	89.77	11.71	17.45	93.93	3.16	6.75	97.03	2.05	4.15	97.26	1.10	5.06
2.0 MB	94.52	10.19	14.84	92.08	1.10	6.02	98.14	0.85	4.27	98.09	0.78	4.98
2.5 MB	95.13	7.86	15.57	93.76	1.13	5.86	98.21	0.63	3.64	98.38	0.62	4.87
3.0 MB	95.96	6.31	12.47	94.34	1.67	5.74	98.72	0.49	3.55	98.63	0.38	4.98

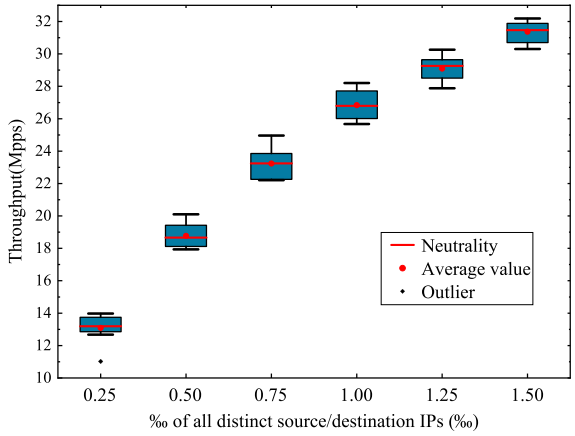
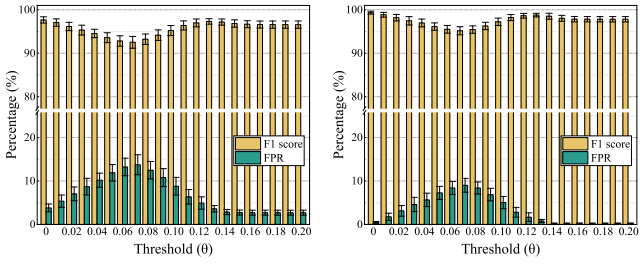


Fig. 14. Sensitivity analysis of the threshold T_e.



(a) Memory size = 0.5 MB (b) Memory size = 1 MB

Fig. 15. Sensitivity analysis of the threshold θ.

Impact of the Lossless Extension Strategy. Disabling lossless extension causes unstable RE across memory sizes (9.57% at 0.5 MB vs. 5.74% at 3.0 MB), indicating reduced adaptability to memory constraints. When the memory size is 0.5 MB, the w/o LE variant even achieves higher FPR than UnivCar (6.23% vs. 7.48%), while getting lower F1 score (79.76%). In our experiments, we also observe that the w/o LE variant achieves high detection precision but its recall rate is no higher than 92%. This demonstrates that lots of previous recorded information of some super hosts is lost without the losses extension strategy. Only those super hosts with very high cardinalities could be detected by the w/o LE variant.

In summary, the combination of the hierarchical data structure and the lossless extension strategy makes UnivCar robust to detect diverse super hosts among different memory constraints, especially when the memory size is below 1.5 MB. For instance, UnivCar and UnivCar-flash respectively achieve 91.61% and 94.45% F1 score at 0.5 MB, outperforming all variants. Besides, the filtering part contributes more in comprehensively improving the detection accuracy, while the lossless extension strategy providing guarantee of the detection recall.

7. Discussion

7.1. Scalability and deployment in hardware-constrained environments

Sketch-based network monitoring solutions employ compact data structures to store statistical features of massive flows, achieving theoretically guaranteed accuracy within limited memory. As a sketch-based super host detection scheme, UnivCar inherently balances memory consumption and detection accuracy, yet encounters the following trade-offs in hardware-constrained environments like smartNICs [64].

Adaptive Extension vs. Memory Overhead. The general extension strategy based on Theorem 2 relies on dynamically allocating additional memory to enable scalable cardinality estimation. While this strategy enhances flexibility in resource-rich environments, it introduces memory overhead that may exceed the capacity of devices like smartNICs. To address this problem, UnivCar is supposed to adopt the lightweight basic bitmap and bucket extensions for deployment in such hardware-constrained environments, sacrificing some extensibility to prioritize memory efficiency.

Skewed Distribution vs. Resource Allocation. UnivCar adopts a hierarchical data structure to optimize memory utilization for highly skewed cardinality distributions by adaptively allocating resources to high cardinalities estimation. However, the filtering part of UnivCar demands non-deductible memory to handle large amounts of low-cardinality flows. When the memory resources are extremely limited, the filtering part will consume a higher proportion of memory. This may reduce the accuracy of detecting super host, necessitating a trade-off between detection sensitivity and resource consumption.

To further enhance deployment flexibility of UnivCar, we plan to explore dynamic resource allocation algorithms that adjust its hierarchical data structure and extension strategy in real time based on hardware constraints. Besides, hybrid approaches combining sketch-based estimation with lightweight hardware acceleration could mitigate current limitations.

7.2. Applicability in encrypted and privacy-preserving networks

Although UnivCar achieves high accuracy on plaintext traffic, its applicability in encrypted networks remains an open challenge. This limitation is prominent in modern network environments like IoT and cloud computing, where large amounts of traffic employs encryption to avoid privacy leakage [72]. In this case, encrypted traffic identification information (e.g., source/destination IP) may hinder UnivCar's tracking of host cardinalities. Recently presented privacy-preserving architecture offers potential solutions. GuardHealth [41] models the topology in a Blockchain based data sharing system to enable malicious node detection based on graph neural network. It maintains the relational topology features required for anomaly detection while avoiding the risk of privacy leakage. However, these privacy-enhancing techniques may introduce additional latency and overhead. Therefore, quantifying the trade-off between privacy assurance and detection response can be one of the directions for future work.

8. Conclusion

In this paper, we propose an extensible and reversible sketch, named UnivCar, for identifying diverse super hosts characterized by high-level or giant changes of cardinalities. UnivCar supports simultaneous and flexible estimation of various cardinalities for large-scale network traffic with skewed cardinality distribution. According to our theoretical analysis and experimental evaluation, UnivCar achieves higher performance and accuracy in identifying super hosts compared to existing solutions. In summary, Univcar serves as a threshold-based network measurement solution for host-related anomaly identification in large-scale networks. Since its detecting efficiency relies on the threshold configuration, future works could focus on establishing adaptive thresholds to guarantee detection accuracy and performance in dynamic network environments.

CRediT authorship contribution statement

Jiqiang Xia: Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization. **Le Tian:** Writing – review & editing, Resources, Methodology, Investigation. **Yuxiang Hu:** Supervision, Investigation, Funding acquisition, Conceptualization. **Jianjin Zhao:** Writing – review & editing, Methodology, Investigation, Formal analysis. **Penghao Sun:** Writing – review & editing, Investigation, Funding acquisition, Formal analysis. **Jianhua Peng:** Writing – review & editing, Supervision, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Science and Technology Innovation Leading Talents Subsidy Project of Central Plains under Grant 244200510038; in part by the Young Elite Scientists Sponsorship Program by CAST under Grant 2023QNRC001; and in part by the National Key Research and Development Program of China under Grant 2022YFB2901501.

Data availability

Data will be made available on request.

References

[1] T. Voronov, D. Raz, O. Rottenstreich, A framework for anomaly detection in blockchain networks with sketches, IEEE/ACM Trans. Netw. 32 (1) (2024) 686–698, http://dx.doi.org/10.1109/TNET.2023.3298253.

[2] H. Han, Z. Yan, X. Jing, W. Pedrycz, Applications of sketches in network traffic measurement: A survey, Inf. Fusion 82 (2022) 58–85, http://dx.doi.org/10.1016/j.inffus.2021.12.007, URL https://www.sciencedirect.com/science/article/pii/S1566253521002578.

[3] D. Ding, M. Savi, F. Pederzolli, M. Campanella, D. Siracusa, In-network volumetric ddos victim identification using programmable commodity switches, IEEE Trans. Netw. Serv. Manag. 18 (2) (2021) 1191–1202, http://dx.doi.org/10.1109/TNSM.2021.3073597.

[4] Z. Liu, H. Namkung, G. Nikolaidis, J. Lee, C. Kim, X. Jin, V. Braverman, M. Yu, V. Sekar, Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, 2021, pp. 3829–3846, URL https://www.usenix.org/conference/usenixsecurity21/presentation/liu-zaoxing.

[5] L. Tang, Y. Xiao, Q. Huang, P.P.C. Lee, A high-performance invertible sketch for network-wide superspreader detection, IEEE/ACM Trans. Netw. 31 (2) (2023) 724–737, http://dx.doi.org/10.1109/TNET.2022.3198738.

[6] Q. Zhang, Q. Xiao, Y. Cai, A generic sketch for estimating super-spreaders and per-flow cardinality distribution in high-speed data streams, Comput. Netw. 237 (C) (2024) http://dx.doi.org/10.1016/j.comnet.2023.110059, URL https://doi.org/10.1016/j.comnet.2023.110059.

[7] H. Han, X. Jing, Z. Yan, W. Pedrycz, ExtendedSketch+: Super host identification and network host trust evaluation with memory efficiency and high accuracy, Inf. Fusion 92 (2023) 300–312, http://dx.doi.org/10.1016/J.INFFUS.2022.12.009, URL https://doi.org/10.1016/j.inffus.2022.12.009.

[8] X. Jing, H. Han, Z. Yan, W. Pedrycz, SuperSketch: A multi-dimensional reversible data structure for super host identification, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2741–2754, http://dx.doi.org/10.1109/TDSC.2021.3072295.

[9] B. Xu, Y. Gong, X. Geng, Y. Li, C. Dong, S. Liu, Y. Liu, B.-S. Jiang, Z. Lu, Procsage: an efficient host threat detection method based on graph representation learning, Cybersecur. 7 (2024) 51, URL https://api.semanticscholar.org/CorpusID:272141105.

[10] T. Schwabe, M. Acosta, Cardinality estimation over knowledge graphs with embeddings and graph neural networks, Proc. ACM Manag. Data 2 (1) (2024) http://dx.doi.org/10.1145/3639299, URL https://doi.org/10.1145/3639299.

[11] P. Wang, Y. Liu, Z. Li, R. Li, An LDP compatible sketch for securely approximating set intersection cardinalities, Proc. ACM Manag. Data 2 (1) (2024) http://dx.doi.org/10.1145/3639281, URL https://doi.org/10.1145/3639281.

[12] H. Wang, D. Melissourgos, C. Ma, S. Chen, Real-time spread burst detection in data streaming, Proc. ACM Meas. Anal. Comput. Syst. 7 (2) (2023) http://dx.doi.org/10.1145/3589979, URL https://doi.org/10.1145/3589979.

[13] Z. Liang, Y. Rong, C. Li, Y. Zhang, Y. Huang, T. Xu, X. Ding, J. Huang, Unsupervised large-scale social network alignment via cross network embedding, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, CIKM '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1008–1017, http://dx.doi.org/10.1145/3459637.3482310, URL https://doi.org/10.1145/3459637.3482310.

[14] Q. Xiao, S. Chen, Y. Zhou, J. Luo, Estimating cardinality for arbitrarily large data stream with improved memory efficiency, IEEE/ACM Trans. Netw. 28 (2) (2020) 433–446, http://dx.doi.org/10.1109/TNET.2020.2970860.

[15] Google Trends, 2024, URL http://www.google.com/trends/. (Accessed 8 October 2024).

[16] L. Rosa, L. Foschini, A. Corradi, Empowering cloud computing with network acceleration: A survey, IEEE Commun. Surv. Tutor. 26 (4) (2024) 2729–2768, http://dx.doi.org/10.1109/COMST.2024.3377531.

[17] S. Tang, Y. Yu, H. Wang, G. Wang, W. Chen, Z. Xu, S. Guo, W. Gao, A survey on scheduling techniques in computing and network convergence, IEEE Commun. Surv. Tutor. 26 (1) (2024) 160–195, http://dx.doi.org/10.1109/COMST.2023.3329027.

[18] J. Zhao, Q. Li, Y. Hong, M. Shen, MetaRockETC: Adaptive encrypted traffic classification in complex network environments via time series analysis and meta-learning, IEEE Trans. Netw. Serv. Manag. 21 (2) (2024) 2460–2476, http://dx.doi.org/10.1109/TNSM.2024.3350080.

[19] B. Claise, Cisco Systems Netflow Services Export Version 9, 3954, RFC Editor, 2004, pp. 1–33, http://dx.doi.org/10.17487/RFC3954.

[20] M. Wang, B. Li, Z. Li, Sflow: Towards resource-efficient and agile service federation in service overlay networks, in: 24th International Conference on Distributed Computing Systems, 2004. Proceedings, IEEE, 2004, pp. 628–635, http://dx.doi.org/10.1109/ICDCS.2004.1281630.

[21] K. Yang, Y. Wu, R. Miao, T. Yang, Z. Liu, Z. Xu, R. Qiu, Y. Zhao, H. Lv, Z. Ji, et al., ChameleMon: Shifting measurement attention as network state changes, in: Proceedings of the ACM SIGCOMM 2023 Conference, 2023, pp. 881–903.

[22] K. Yang, S. Long, Q. Shi, Y. Li, Z. Liu, Y. Wu, T. Yang, Z. Jia, Sketchint: Empowering INT with TowerSketch for per-flow per-switch measurement, IEEE Trans. Parallel Distrib. Syst. 34 (11) (2023) 2876–2894, http://dx.doi.org/10.1109/TPDS.2023.3303924.

[23] X. Song, J. Zheng, H. Qian, S. Zhao, H. Zhang, X. Pan, G. Chen, Couper: Memory-efficient cardinality estimation under unbalanced distribution, in: 2023 IEEE 39th International Conference on Data Engineering, ICDE, 2023, pp. 2753–2765, http://dx.doi.org/10.1109/ICDE55515.2023.00211.

[24] W. Liu, W. Qu, J. Gong, K. Li, Detection of superpoints using a vector bloom filter, IEEE Trans. Inf. Forensics Secur. 11 (3) (2016) 514–527, http://dx.doi.org/10.1109/TIFS.2015.2503269.

[25] H. Yue, T. Li, Di Wu, R. Zhang, Z. Yang, Detecting APT attacks using an attack intent-driven and sequence-based learning approach, Comput. Secur. 140 (2024) 103748, http://dx.doi.org/10.1016/j.cose.2024.103748, URL https://www.sciencedirect.com/science/article/pii/S016740482400049X.

[26] S. Chaudhary, P.K. Mishra, Ddos attacks in industrial IoT: A survey, Comput. Netw. 236 (2023) 110015, http://dx.doi.org/10.1016/j.comnet.2023.110015, URL https://www.sciencedirect.com/science/article/pii/S1389128623004607.

[27] A. Khraisat, A. Alazab, S. Singh, T. Jan, A. Gomez Jr., Survey on federated learning for intrusion detection system: Concept, architectures, aggregation strategies, challenges, and future directions, ACM Comput. Surv. 57 (1) (2024) http://dx.doi.org/10.1145/3687124, URL https://doi.org/10.1145/3687124.

[28] Q. Li, H. Huang, R. Li, J. Lv, Z. Yuan, L. Ma, Y. Han, Y. Jiang, A comprehensive survey on ddos defense systems: New trends and challenges, Comput. Netw. 233 (2023) 109895, http://dx.doi.org/10.1016/j.comnet.2023.109895, URL https://www.sciencedirect.com/science/article/pii/S1389128623003407.

[29] A. Zhou, J. Qian, An adaptive method for identifying super nodes from network-wide view, J. Netw. Syst. Manage. 31 (3) (2023) http://dx.doi.org/10.1007/s10922-023-09745-0, URL https://doi.org/10.1007/s10922-023-09745-0.

[30] A. Liotta, G. Pavlou, G. Knight, Exploiting agent mobility for large-scale network monitoring, IEEE Netw. 16 (3) (2002) 7–15, http://dx.doi.org/10.1109/MNET.2002.1002994.

[31] Y. Du, H. Huang, Y.-E. Sun, S. Chen, G. Gao, Self-adaptive sampling for network traffic measurement, in: IEEE INFOCOM 2021 - IEEE Conference on Computer Communications, 2021, pp. 1–10, http://dx.doi.org/10.1109/INFOCOM42981.2021.9488425.

[32] N. Kamiyama, T. Mori, R. Kawahara, Simple and adaptive identification of superspreaders by flow sampling, in: IEEE INFOCOM 2007 - 26th IEEE International Conference on Computer Communications, 2007, pp. 2481–2485, http://dx.doi.org/10.1109/INFCOM.2007.305.

[33] S. Venkataraman, D.X. Song, P.B. Gibbons, A. Blum, New streaming algorithms for fast detection of superspreaders, in: Network and Distributed System Security Symposium, 2005, URL https://api.semanticscholar.org/CorpusID:538509.

[34] J. Cao, Y. Jin, A. Chen, T. Bu, Z.-L. Zhang, Identifying high cardinality internet hosts, in: IEEE INFOCOM 2009, 2009, pp. 810–818, http://dx.doi.org/10.1109/INFCOM.2009.5061990.

[35] H. Huang, Y.-E. Sun, C. Ma, S. Chen, Y. Du, H. Wang, Q. Xiao, Spread estimation with non-duplicate sampling in high-speed networks, IEEE/ACM Trans. Netw. 29 (5) (2021) 2073–2086, http://dx.doi.org/10.1109/TNET.2021.3078725.

[36] C. Ma, H. Wang, O.O. Odegbile, S. Chen, D. Melissourgos, Virtual filter for non-duplicate sampling with network applications, IEEE/ACM Trans. Netw. 30 (6) (2022) 2818–2833, http://dx.doi.org/10.1109/TNET.2022.3182694.

[37] Y. Cheng, X. Qiao, X. Wang, Q. Yu, Random forest classifier for zero-shot learning based on relative attribute, IEEE Trans. Neural Netw. Learn. Syst. 29 (5) (2018) 1662–1674, http://dx.doi.org/10.1109/TNNLS.2017.2677441.

[38] W.H. Turkett, A.V. Karode, E.W. Fulp, In-the-dark network traffic classification using support vector machines, in: Proceedings of the 20th National Conference on Innovative Applications of Artificial Intelligence - Volume 3, IAAI '08, AAAI Press, 2008, pp. 1745–1750.

[39] A. Parameswari, R. Ganeshan, V. Ragavi, M. Shereesha, Hybrid rat swarm hunter prey optimization trained deep learning for network intrusion detection using CNN features, Comput. Secur. 139 (2024) 103656, http://dx.doi.org/10.1016/j.cose.2023.103656, URL https://www.sciencedirect.com/science/article/pii/S0167404823005655.

[40] Y. Xue, C. Kang, H. Yu, HAE-hrl: A network intrusion detection system utilizing a novel autoencoder and a hybrid enhanced LSTM-CNN-based residual network, Comput. Secur. 151 (2025) 104328, http://dx.doi.org/10.1016/j.cose.2025.104328, URL https://www.sciencedirect.com/science/article/pii/S0167404825000173.

[41] Z. Wang, N. Luo, P. Zhou, GuardHealth: Blockchain empowered secure data management and graph convolutional network enabled anomaly detection in smart healthcare, J. Parallel Distrib. Comput. 142 (2020) 1–12, http://dx.doi.org/10.1016/j.jpdc.2020.03.004, URL https://www.sciencedirect.com/science/article/pii/S0743731519308470.

[42] X. Yang, H. Liu, S.A. Tsegai, P. Gao, Zebra: Deeply integrating system-level provenance search and tracking for efficient attack investigation, 2022, arXiv:2211.05403. URL https://arxiv.org/abs/2211.05403.

[43] C. Li, Y. Zhang, J. Li, Y. Huang, X. Ding, Unsupervised anomaly segmentation using image-semantic cycle translation, 2021, arXiv:2103.09094. URL https://arxiv.org/abs/2103.09094.

[44] Z. Wang, Y. Lu, W. Wu, Y. Lu, H. Wang, A few-shot and anti-forgetting network intrusion detection system based on online meta learning, in: GLOBECOM 2024 - 2024 IEEE Global Communications Conference, 2024, pp. 3877–3882, http://dx.doi.org/10.1109/GLOBECOM52923.2024.10901423.

[45] H. Kheddar, D.W. Dawoud, A.I. Awad, Y. Himeur, M.K. Khan, Reinforcement-learning-based intrusion detection in communication networks: A review, IEEE Commun. Surv. Tutor. (2024) http://dx.doi.org/10.1109/COMST.2024.3484491, 1–1.

[46] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T.Y. Hou, M. Tegmark, KAN: Kolmogorov-arnold networks, 2025, arXiv:2404.19756. URL https://arxiv.org/abs/2404.19756.

[47] C. Li, X. Liu, W. Li, C. Wang, H. Liu, Y. Liu, Z. Chen, Y. Yuan, U-KAN makes strong backbone for medical image segmentation and generation, 2024, arXiv:2406.02918. URL https://arxiv.org/abs/2406.02918.

[48] C. Li, Yuzhihuang, W. Li, H. Liu, X. Liu, Q. Xu, Z. Chen, Y. Huang, Y. Yuan, Flaws can be applause: Unleashing potential of segmenting ambiguous objects in SAM, in: The Thirty-Eighth Annual Conference on Neural Information Processing Systems, 2024, URL https://openreview.net/forum?id=vJSNsSFO95.

[49] Y. Huang, C. Li, Z. Lin, H. Liu, H. Xu, Y. Liu, Y. Huang, X. Ding, X. Tu, Y. Yuan, P2SAM: Probabilistically prompted SAMs are efficient segmentator for ambiguous medical images, in: Proceedings of the 32nd ACM International Conference on Multimedia, MM '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 9779–9788, http://dx.doi.org/10.1145/3664647.3680628, URL https://doi.org/10.1145/3664647.3680628.

[50] P. Wang, X. Guan, T. Qin, Q. Huang, A data streaming method for monitoring host connection degrees of high-speed links, IEEE Trans. Inf. Forensics Secur. 6 (3) (2011) 1086–1098, http://dx.doi.org/10.1109/TIFS.2011.2123094.

[51] Y. Du, H. Huang, Y.-E. Sun, K. Li, B. Zhang, G. Gao, A better cardinality estimator with fewer bits, constant update time, and mergeability, in: IEEE INFOCOM 2023 - IEEE Conference on Computer Communications, 2023, pp. 1–10, http://dx.doi.org/10.1109/INFOCOM53939.2023.10229088.

[52] G. Cormode, S. Muthukrishnan, An improved data stream summary: the count-min sketch and its applications, J. Algorithms 55 (1) (2005) 58–75, http://dx.doi.org/10.1016/J.JALGOR.2003.12.001.

[53] C. Estan, G. Varghese, M. Fisk, Bitmap algorithms for counting active flows on high speed links, in: Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement, 2003, pp. 153–166, http://dx.doi.org/10.1145/948205.948225.

[54] K. Yang, Y. Wu, R. Miao, T. Yang, Z. Liu, Z. Xu, R. Qiu, Y. Zhao, H. Lv, Z. Ji, G. Xie, ChameleMon: Shifting measurement attention as network state changes, in: Proceedings of the ACM SIGCOMM 2023 Conference, in: ACM SIGCOMM '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 881–903, http://dx.doi.org/10.1145/3603269.3604850, URL https://doi.org/10.1145/3603269.3604850.

[55] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, V. Braverman, One sketch to rule them all: Rethinking network flow monitoring with UnivMon, in: Proceedings of the 2016 ACM SIGCOMM Conference, SIGCOMM '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 101–114, http://dx.doi.org/10.1145/2934872.2934906, URL https://doi.org/10.1145/2934872.2934906.

[56] T. Yang, J. Jiang, P. Liu, Q. Huang, J. Gong, Y. Zhou, R. Miao, X. Li, S. Uhlig, Elastic sketch: adaptive and fast network-wide measurements, in: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 561–575, http://dx.doi.org/10.1145/3230543.3230544, URL https://doi.org/10.1145/3230543.3230544.

[57] S.J. Kim, C. Jung, R. Jang, D.A. Mohaisen, D. Nyang, A robust counting sketch for data plane intrusion detection, in: Proceedings 2023 Network and Distributed System Security Symposium, 2023, URL https://api.semanticscholar.org/CorpusID:257499995.

[58] M. Yang, J. Zhang, A. Gadre, Z. Liu, S. Kumar, V. Sekar, Joltik: enabling energy-efficient "future-proof" analytics on low-power wide-area networks, in: Proceedings of the 26th Annual International Conference on Mobile Computing and Networking, MobiCom '20, Association for Computing Machinery, New York, NY, USA, 2020, http://dx.doi.org/10.1145/3372224.3419204, URL https://doi.org/10.1145/3372224.3419204.

[59] Q. Xiao, Z. Tang, S. Chen, Universal online sketch for tracking heavy hitters and estimating moments of data streams, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications, 2020, pp. 974–983, http://dx.doi.org/10.1109/INFOCOM41043.2020.9155454.

[60] J. Liu, R.B. Basat, L. De Wardt, H. Dai, G. Chen, DISCO: A dynamically configurable sketch framework in skewed data streams, in: 2024 IEEE 40th International Conference on Data Engineering, ICDE, 2024, pp. 4801–4814, http://dx.doi.org/10.1109/ICDE60146.2024.00365.

[61] C. Estan, G. Varghese, New directions in traffic measurement and accounting, in: Proceedings of the 2002 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, SIGCOMM '02, Association for Computing Machinery, New York, NY, USA, 2002, pp. 323–336, http://dx.doi.org/10.1145/633025.633056, URL https://doi.org/10.1145/633025.633056.

[62] M. Charikar, K. Chen, M. Farach-Colton, Finding frequent items in data streams, Theoret. Comput. Sci. 312 (1) (2004) 3–15, http://dx.doi.org/10.1016/S0304-3975(03)00400-6, Automata, Languages and Programming. URL https://www.sciencedirect.com/science/article/pii/S0304397503004006.

[63] T. Qin, X. Guan, W. Li, P. Wang, M. Zhu, A new connection degree calculation and measurement method for large scale network monitoring, J. Netw. Comput. Appl. 41 (2014) 15–26, http://dx.doi.org/10.1016/j.jnca.2013.10.008, URL https://www.sciencedirect.com/science/article/pii/S1084804513002130.

[64] E.F. Kfoury, S. Choueiri, A. Mazloum, A. AlSabeh, J. Gomez, J. Crichigno, A comprehensive survey on smartnics: Architectures, development models, applications, and research directions, IEEE Access 12 (2024) 107297–107336, http://dx.doi.org/10.1109/ACCESS.2024.3437203.

[65] B. Jenkins, A hash function for hash table lookup, 2006, URL http://burtleburtle.net/bob/hash/doobs.html.

[66] A. Appleby, MurmurHash: A non-cryptographic hash function, 2008, Initial version released in 2008; MurmurHash3 updated in 2012. https://sites.google.com/site/murmurhash/.

[67] M. Mitzenmacher, E. Upfal, Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis, second ed., Cambridge University Press, 2017.

[68] L. Luo, D. Guo, R.T.B. Ma, O. Rottenstreich, X. Luo, Optimizing bloom filter: Challenges, solutions, and comparisons, IEEE Commun. Surv. Tutor. 21 (2) (2019) 1912–1949, http://dx.doi.org/10.1109/COMST.2018.2889329.

[69] M. Molina, S. Niccolini, N. Duffield, C. Dante, A comparative experimental study of hash functions applied to packet sampling, 2005, pp. 1–10.

[70] C. Guang, Z. Wei, G. Jian, XOR hashing algorithms to measured flows at the high-speed link, in: 2008 Second International Conference on Future Generation Communication and Networking, vol. 1, 2008, pp. 152–155, http://dx.doi.org/10.1109/FGCN.2008.110.

[71] The CAIDA UCSD anonymized internet traces 2018, 2023, URL http://www.caida.org/data/passive/passive_2018_dataset.xml. (Accessed 8 November 2023).

[72] Y. Yao, Z. Wang, P. Zhou, Privacy-preserving and energy efficient task offloading for collaborative mobile computing in IoT: An ADMM approach, Comput. Secur. 96 (2020) 101886, http://dx.doi.org/10.1016/j.cose.2020.101886, URL https://www.sciencedirect.com/science/article/pii/S0167404820301590.